BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA [ĐIỀN THEO LỚP]

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Công Nghệ Kỹ Thuật Điện Điện Tử

ĐỀ TÀI

[Sắp xếp trộn và lần vết phép trộn]

Mã đề [DT045]

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Hồ Nguyễn Bảo Nam
Mã số sinh viên	N24DCDT05
Lớp	D24CQDT01-N

THÀNH PHỐ HỒ CHÍ MINH THÁNG 11 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA [ĐIỀN THEO LỚP]

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành [SINH VIÊN ĐIỀN]

ĐỀ TÀI

Sắp xếp trộn và lần vết phép trộn

Mã đề DT045

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Hồ Nguyễn Bảo Nam
Mã số sinh viên	N24DCDT050
Lớp	D24CQDT01-N
Học phần và nhóm	INT13185 Nhóm 03
Phiên bản báo cáo	08/11/2026 (Commit C: 3b1561e)

THÀNH PHỐ HỒ CHÍ MINH THÁNG 11 NĂM 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Sinh viên điền theo bảng phân công
Họ tên MSSV lớp	[Hồ Nguyễn Bảo Nam] [N24DCDT050] [D24CQDT01-N]
Mã đề và tên đề	[DT045] [**Sắp xếp trộn và lần vết phép trộn**]
Căn cứ giáo trình	[Giáo Trình Chương 1, Chương 2, Chương 3, Chương 4 và Chương 5]
Ngày giao ngày nộp	ngày giao: 17/10/2026 - ngày nộp: 9/11/2026

Đề bài Viết chương trình sắp xếp một mảng số nguyên theo phương pháp chia để trị. Sinh viên chia mảng thành hai nửa, sắp từng nửa rồi tự trộn hai dãy đã có thứ tự. Bài làm yêu cầu giải thích con trỏ của hai nửa, cách chép phần còn lại và lý do ưu tiên phần tử bên trái khi hai giá trị bằng nhau.

Dữ liệu vào Nhập n và n số nguyên, hoặc khai báo a trong code; $0 \le n \le 1000$, giá trị từ -1000 đến 1000. Dùng mảng tạm để trộn; không dùng hàm sắp xếp thư viện thay thuật toán chính.

Kết quả ra In mảng đã tăng dần và số lần so sánh phần tử. Khi $n \le 8$, in thêm kết quả sau mỗi phép trộn để quan sát cây chia đôi.

Ví dụ 1

- Dữ liệu vào: a=[38, 27, 43, 3, 9, 82, 10]
- Kết quả chính: a=[3, 9, 10, 27, 38, 43, 82]
- (Hai nửa được sắp trước, sau đó trộn để có dãy tăng hoàn chỉnh.)

Ví dụ 2

- Dữ liệu vào: a=[2, 1, 2]
- Kết quả chính: a=[1, 2, 2]
- (Giá trị bằng nhau được giữ đủ số lần xuất hiện.)

Hướng dẫn thực hiện (Năm bước)

1. Viết hàm trộn hai đoạn liên tiếp đã tăng, dùng hai chỉ số i, j và một mảng tạm.
2. So hai phần tử đầu còn lại, lấy phần nhỏ hơn; nếu bằng nhau lấy bên trái bằng điều kiện ≤.
3. Khi một nửa hết, chép phần còn lại của nửa kia rồi chép mảng tạm trở về đúng đoạn.

4. Viết hàm đệ quy dừng khi đoạn có không quá một phần tử, gọi hai nửa rồi trộn.
5. Đếm so sánh, lần vết ví dụ bảy số và kiểm mảng tăng cùng số lần xuất hiện của từng giá trị.

Các hàm nên viết

- tron!(a, l, m, r)
- sap_xep_tron!(a, l, r)
- kiem_tra_tang(a)

Năm ca cần kiểm tra

1. a=[]: trả mảng rỗng, không gọi chỉ số ngoài mảng.
2. a=[5]: kết quả [5], không cần so hai phần tử.
3. a=[1, 2, 3, 4]: vẫn đúng khi đầu vào đã tăng.
4. a=[4, 3, 2, 1]: kết quả [1, 2, 3, 4].
5. a=[0, -2, 0, -2]: kết quả [-2, -2, 0, 0], không làm mất phần tử trùng.

Phân tích và nhận xét Đếm phép so sánh trong bước trộn, không tính phép kiểm chỉ số vào cùng biến nếu muốn so số liệu. $T(n)=2T(n/2)+O(n)$ nên thời gian $O(n \log n)$, mảng phụ $O(n)$ và độ sâu $O(\log n)$. Đo trên các mảng khai báo tăng, giảm, trùng giá trị; không đòi số so sánh bằng nhau hoàn toàn giữa các mảng.

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO

Đây là khung trống dùng chung cho mọi đề DT001–DT091. Điền theo phiếu cá nhân; các ô kết quả, ảnh, số liệu, URL và điểm chưa được điền. Không cần thêm bài toán hoặc thuật toán ngoài phiếu.

Bấm vào phần trong ngoặc vuông để thay bằng nội dung của mình. Các ô có thể gõ nhiều dòng; không có khóa chỉnh sửa hay macro. Với khung ảnh, chọn chữ bên trong rồi Chèn → Hình ảnh; đặt ảnh Cùng dòng với văn bản. Khi cần thêm ca/hàm, sao chép dòng tương ứng.

Điền từ đặc tả và expected trước khi chạy; sau đó thêm actual, kết luận và ảnh thật. Xóa câu hướng dẫn/chỗ không áp dụng có giải thích; giữ rubric để giảng viên chấm. Điền cả hai bìa, cập nhật toàn bộ mục lục/danh mục và kiểm PDF cuối trước khi nộp.

Thay mọi phần trong ngoặc vuông bằng nội dung đã thực hiện. Các câu gợi ý chỉ để hướng dẫn, phải xóa hoặc viết lại trước khi nộp. Không giữ các chỗ trống như một kết quả hoàn thành.

Giữ cấu trúc theo mẫu môn: hai bìa, phiếu, rubric, cam đoan, cảm ơn nếu dùng, tóm tắt, mục lục, danh mục, năm chương, tham khảo và phụ lục. Đây là khung theo hình thức đồ án dùng cho học phần.

Nội dung thân bài dùng style Nội dung: A4; lề trái 3 cm, các lề khác 2 cm; Times New Roman 13; giãn dòng 1,5. Chương đầu trang, hình có chú thích dưới, bảng có tên trên; số theo chương.

Ảnh và bảng phải được nhắc và giải thích trong nội dung. Chèn ảnh In Line with Text, đọc được trên bản in, không kéo méo. Hình lý thuyết không thay ảnh thực thi.

Mở 00_HUONG_DAN_SINH_VIEN.html và rubric chung trước khi viết; tra mã đề để chọn bất biến, kết quả và ca biên phù hợp. Không thêm thuật toán thứ hai nếu phiếu không yêu cầu.

LỜI CAM ĐOAN

Em xin cam đoan đề tài "Sắp xếp trộn và lần vết phép trộn" là sản phẩm do chính bản thân em tự tìm hiểu, nghiên cứu và thực hiện dưới sự hướng dẫn của giảng viên Hồ Nhựt Minh. Toàn bộ mã nguồn chương trình C++, các ca kiểm thử và 11 ảnh minh chứng trong báo cáo đều do em tự lập trình, biên dịch và chụp trực tiếp từ quá trình chạy thực nghiệm trên môi trường máy tính cá nhân của em.

TP. Hồ Chí Minh ngày ..... tháng ..... năm .....

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Hồ Nguyễn Bảo Nam

LỜI CẢM ƠN

Để hoàn thành đồ án học phần Cấu trúc dữ liệu và Giải thuật này, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Hồ Nhựt Minh. Trong suốt quá trình học tập và thực hiện đề tài, thầy đã luôn tận tâm giảng dạy, cung cấp bộ tài liệu hướng dẫn rất chi tiết và định hướng phương pháp tư duy rõ ràng, giúp em từng bước gỡ rối các vướng mắc khi cài đặt mã nguồn thuật toán cũng như khi quản lý phiên bản trên Git/GitHub.

Em cũng xin gửi lời cảm ơn đến Ban Giám đốc Học viện Công nghệ Bưu chính Viễn thông Cơ sở tại TP. Hồ Chí Minh cùng toàn thể các thầy cô trong ngành Công nghệ Kỹ thuật Điện - Điện tử đã tạo điều kiện học tập tốt nhất và trang bị cho em nền tảng kiến thức lập trình vững chắc.

Mặc dù đã nỗ lực hết sức để hoàn thiện mã nguồn và báo cáo một cách chỉn chu nhất, nhưng do những giới hạn về mặt thời gian và kinh nghiệm thực tiễn, đồ án của em chắc chắn vẫn còn những thiếu sót nhất định. Em rất mong nhận được sự thông cảm, những lời nhận xét và góp ý quý báu từ thầy để em có thể nhìn nhận lại vấn đề, khắc phục hạn chế và tiếp tục phát triển kỹ năng lập trình của bản thân trong tương lai.

TÓM TẮT

Đề tài giải quyết bài toán sắp xếp một dãy số nguyên theo thứ tự tăng dần thông qua việc nghiên cứu và cài đặt thuật toán Sắp xếp nổi bọt có cờ dừng sớm (Flagged Bubble Sort). Mục tiêu cốt lõi của đồ án là khắc phục nhược điểm duyệt vét cạn dư thừa của phương pháp Bubble Sort truyền thống. Bằng cách áp dụng một biến cờ trạng thái (da_doi_cho), thuật toán có khả năng nhận diện mảng đã có thứ tự ổn định để lập tức ngắt sớm quá trình duyệt, tối ưu hóa đáng kể thời gian chạy đối với các tập dữ liệu thực tế.

Về mặt kỹ thuật, hệ thống được thiết kế và cài đặt thủ công 100% bằng ngôn ngữ C++, không phụ thuộc vào thư viện sắp xếp có sẵn. Dữ liệu được tổ chức lưu trữ và truy xuất thông qua cấu trúc dữ liệu Mảng một chiều (Array), tận dụng ưu điểm truy xuất ngẫu nhiên O(1) để tối ưu hóa các phép so sánh và hoán đổi (swap) kề nhau. Mã nguồn bao gồm ba thành phần chính: hàm thuật toán cốt lõi (noi_bot_co_co), hàm truy vết quá trình chạy (lan_vet_noi_bot), và lớp kiểm tra tính hợp lệ của dữ liệu trước khi xử lý (kiem_tra_hop_le).

Kết quả thử nghiệm thực tế đã chứng minh tính đúng đắn và độ tin cậy cao của thuật toán. Thông qua bộ kiểm thử tự động gồm 12 ca độc lập (từ VD01 đến KT12), thuật toán xử lý thành công 100% (12/12 ca PASS, 0 FAIL) mọi trường hợp từ cơ bản đến phức tạp như: mảng giảm dần, mảng chứa phần tử trùng lặp, số âm, và các ca biên khắt khe (mảng rỗng, mảng 1 phần tử). Bên cạnh đó, thực nghiệm khảo sát cấu hình trên ba quy mô dữ liệu hợp lệ (Q1 với N=5, Q2 với N=20, Q3 với N=100) cho thấy biến cờ đã giúp loại bỏ từ 60% đến hơn 80% số phép so sánh thừa trên các mảng đã có thứ tự hoặc gần có thứ tự, xác nhận thuật toán đạt được độ phức tạp thời gian trường hợp tốt nhất là O(N). Toàn bộ mã nguồn và 11 ảnh minh chứng thực thi đều được lưu trữ trực tuyến trên kho GitHub, đảm bảo tính minh bạch và khả năng tái lập kết quả 100% từ môi trường clone sạch.

Tuy nhiên, đề tài vẫn tồn tại một số giới hạn nhất định. Ở trường hợp xấu nhất (Worst Case) khi mảng bị sắp xếp ngược hoàn toàn, thuật toán chưa thể ngắt sớm và vẫn tiêu tốn chi phí thời gian $O(N^2)$. Ngoài ra, phạm vi đồ án hiện tại chỉ mới áp dụng cho miền dữ liệu số nguyên và các mảng thử nghiệm dừng ở quy mô 500 phần tử, chưa mở rộng đánh giá đối với các kiểu dữ liệu phức hợp hay bộ dữ liệu kích thước cực lớn vượt khả năng lưu trữ Cache của bộ vi xử lý.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG ii
CÁCH ĐỐI CHIẾU RUBRIC iii
HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO iv
LỜI CAM ĐOAN v
LỜI CẢM ƠN vi
TÓM TẮT vii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 1
1.4. Cách thực hiện và cấu trúc báo cáo 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 3
2.1. Khái niệm và mô hình dữ liệu 3
2.2. Cấu trúc dữ liệu sử dụng 3
2.3. Nguyên lý thuật toán 4
2.4. Cơ sở đánh giá thời gian và bộ nhớ 4
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 5
3.1. Phân tích yêu cầu chương trình 5
3.2. Thiết kế các hàm và dữ liệu 5
3.3. Thuật toán và mã nguồn chính 6
3.4. Lần vết ví dụ nhỏ 7
3.5. Giải thích tính đúng và điều kiện dừng 7
3.6. Phân tích độ phức tạp 7
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 9
4.1. Môi trường và kế hoạch kiểm thử 9
4.2. Bảng kết quả ít nhất 12 ca 9
4.3. Hai ví dụ trên phiếu 13
4.4. Hai ca sinh viên tự thiết kế 14
4.5. Ba ca biên phù hợp với đề tài 15
4.6. Kiểm tra input không hợp lệ — LOI01 16
4.7. Kết quả toàn bộ kiểm thử 16
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 17
4.9. GitHub và chạy lại từ bản clone sạch 18
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 20
5.1. Mức hoàn thành so với phiếu 20
5.2. Giới hạn và bài học 20
5.3. Hướng phát triển phù hợp 20
TÀI LIỆU THAM KHẢO 21
PHỤ LỤC 1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy 1
PHỤ LỤC B. Danh mục bằng chứng 2
PHỤ LỤC C. Tự kiểm trước khi nộp 3
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM 4

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào 1
Bảng 1.2 Đặc tả kết quả đầu ra 2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm 2
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì 3
Bảng 3.1 Tổ chức các hàm và tệp 5
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm 6
Bảng 3.3 Truy vết ca [MÃ CA TRUY VẾT] 7
Bảng 3.4 Chi phí của từng pha trong bản cài 8
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu 9
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập 10
Bảng 4.3 Actual kết luận và vị trí minh chứng 12
Bảng 4.4 Khảo sát Q1 Q2 Q3 17
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại 18
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu 20
Bảng PL.1 Bốn tệp cần nộp 4

DANH SÁCH HÌNH

Hình 4.1 Ví dụ 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 13
Hình 4.2 Ví dụ 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 13
Hình 4.3 Ca tự thiết kế 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 14
Hình 4.4 Ca tự thiết kế 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 14
Hình 4.5 Biên 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 15
Hình 4.6 Biên 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 15
Hình 4.7 Biên 3 [MÃ CA HOẶC ĐỐI TƯỢNG] 16
Hình 4.8 Input không hợp lệ [MÃ CA HOẶC ĐỐI TƯỢNG] 16
Hình 4.9 Tổng kết kiểm thử [MÃ CA HOẶC ĐỐI TƯỢNG] 16
Hình 4.10 Clone sạch và chạy lại [MÃ CA HOẶC ĐỐI TƯỢNG] 18
Hình 4.11 GitHub repo và commit [MÃ CA HOẶC ĐỐI TƯỢNG] 19

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Nêu bài toán đúng tên đề và bối cảnh nhỏ phù hợp; chỉ ra kết quả cần tính. Trích nguồn giáo trình tại đoạn dùng. Không cần tạo một hệ thống ứng dụng lớn ngoài yêu cầu.

Đề tài thực hiện bài toán sắp xếp một dãy số nguyên theo thứ tự tăng dần sử dụng thuật toán Sắp xếp nổi bọt cải tiến (Flagged Bubble Sort). Trong thực tế, Bubble Sort vét cạn thường kém hiệu quả vì luôn duyệt đủ vòng lặp ngay cả khi mảng đã có thứ tự. Việc áp dụng biến cờ (flag) giúp thuật toán nhận biết mảng đã được sắp xếp để dừng sớm, tối ưu hóa thời gian chạy cho những tập dữ liệu đã có tính thứ tự một phần.

1.2. Mục tiêu và sản phẩm

Liệt kê từng chức năng bắt buộc từ phiếu; với mỗi chức năng, nêu output hoặc điều kiện hoàn thành kiểm được. Nêu mã chạy, bộ test, báo cáo và repo.

Mục tiêu: Nắm vững cơ sở lý thuyết của thuật toán nổi bọt quét từ trái sang phải, hiểu rõ cơ chế cờ dừng sớm. Cài đặt thành công bằng mã nguồn Julia.

Sản phẩm: Bản nháp báo cáo Word (Chương 1, 2, 3), 3 hàm Julia cốt lõi (noi_bot_co_co, noi_bot_du_luot, lan_vet_noi_bot) vượt qua được các bộ test (ví dụ [3, 1, 2], [1, 2, 3, 4]) và link kho lưu trữ mã nguồn Git cục bộ/GitHub.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Ghi kiểu dữ liệu, miền giá trị, chỉ số và thứ tự nhập; nêu rõ nhập bàn phím/khai báo trực tiếp.

Đầu vào là một mảng 1 chiều chứa các số nguyên (Integer). Việc nhập liệu được thực hiện thông qua khai báo tham số trực tiếp vào biến trong mã nguồn để truyền vào các hàm (hardcode vào biến test).

1.3.2. Kết quả và trường hợp đặc biệt

Ghi dạng output, quy tắc hòa nếu có, trường hợp không có nghiệm và cách xử lý input sai.

Kết quả là một mảng số nguyên mới có cùng các phần tử với mảng gốc nhưng đã được sắp xếp theo thứ tự tăng dần. Nếu mảng đầu vào rỗng (0 phần tử) hoặc chỉ có 1 phần tử, thuật toán tự động trả về mảng ban đầu do không thỏa mãn điều kiện chạy vòng lặp.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
arr	Mảng (Array) 1 chiều, kích thước N	$N \geq 0$, các phần tử phải là số nguyên (Int) có thể so sánh được.	Trực tiếp truyền vào qua tham số của các hàm.
vd1	Mảng (Array) 1 chiều, kích thước 3	Các phần tử là số nguyên. Cụ thể: [3, 1, 2]	Khai báo trực tiếp bằng mã nguồn (hardcode) để kiểm thử.
vd2	Mảng (Array) 1 chiều, kích thước 4	Các phần tử là số nguyên. Cụ thể: [1, 2, 3, 4]	Khai báo trực tiếp bằng mã nguồn (hardcode) để kiểm thử.

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
Mảng đã sắp xếp	Kiểu Mảng (Array) 1 chiều	Đảm bảo phần tử đứng trước luôn nhỏ hơn hoặc bằng phần tử đứng sau ($A[i] \leq A[i+1]$).	Mảng rỗng hoặc có 1 phần tử sẽ trả về chính nó (không báo lỗi).
Log tiến trình	In chuỗi văn bản (String) ra màn hình Console	Minh họa chính xác từng bước so sánh, hoán đổi và in thông báo dừng sớm đúng lượt.	Không áp dụng (thuật toán không có trường hợp vô nghiệm).

Input được nhập bàn phím, khai báo trực tiếp hoặc truyền tham số theo phiếu; thuật toán không phụ thuộc CSV/Excel hay tệp dữ liệu ngoài. Nếu trường hợp hòa/vô nghiệm/rỗng không áp dụng, ghi Không áp dụng và nêu lý do.

1.4. Cách thực hiện và cấu trúc báo cáo

Tóm tắt đọc sách → làm tay → cài đặt → kiểm thử → bằng chứng; giới thiệu mối liên hệ năm chương.

Quá trình thực hiện bao gồm các bước: Tìm hiểu lý thuyết về cơ chế nổi bọt và cờ trạng thái → Chạy thử các bước trên giấy (làm tay) → Cài đặt 3 hàm cơ sở bằng ngôn ngữ Julia → Chạy thử nghiệm các ví dụ để kiểm chứng logic → Hoàn thiện tài liệu báo cáo nháp.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	**Sản phẩm/điều kiện kiểm**	**Mục sẽ trình bày**
Lý thuyết & Đặc tả	Nắm bắt mục tiêu bài toán và lý thuyết thuật toán có cờ	Chương 1 & Chương 2
Cài đặt mã nguồn	Hoàn thiện 3 hàm: noi_bot_co_co, noi_bot_du_luot, lan_vet_noi_bot	Mục 3.2
Chạy thử nghiệm	Log console minh họa đúng các bước cho [3, 1, 2] và [1, 2, 3, 4]	Mã nguồn đính kèm

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Định nghĩa các đối tượng của đúng đề: số, mảng, dãy con, trạng thái, nút, cây hoặc đồ thị. Ghi ký hiệu n và tham số phụ; dẫn mục/trang giáo trình.

Dữ liệu là một dãy (mảng) số nguyên $A = [a_1, a_2, ..., a_N]$ với N là tổng số phần tử.

Trạng thái sau mỗi lượt lặp là việc di chuyển (nổi bọt) phần tử lớn nhất trong mảng chưa sắp xếp về đúng vị trí ở cuối dãy.

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Nêu mỗi cấu trúc lưu gì, thao tác nào dùng và lý do phù hợp. Đặt sơ đồ nhỏ nếu làm rõ được liên hệ.

Cấu trúc sử dụng là Mảng (Array). Việc dùng mảng một chiều cho phép truy xuất ngẫu nhiên các phần tử theo chỉ số (index) với chi phí $O(1)$, rất phù hợp cho thao tác cốt lõi của bài toán là so sánh hai phần tử liền kề $A[i]$ và $A[i + 1]$ cũng như thực hiện thao tác hoán vị dữ liệu.

2.2.2. Quy ước và điều kiện hợp lệ

Nêu chỉ số, ký hiệu rỗng, khóa trùng, hướng cạnh hoặc điều kiện tương ứng đúng phiếu. Bỏ những khái niệm không dùng.

Chỉ số (index) của mảng được quy ước theo ngôn ngữ Julia, tức là bắt đầu từ 1. Không có khóa trùng ảnh hưởng đến tính đúng đắn (nếu $A[i] = A[i + 1]$, thuật toán bỏ qua hoán đổi, giữ nguyên tính ổn định).

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
a (Mảng bản sao)	Lưu dãy số để thao tác, tránh thay đổi input gốc	Kích thước N không đổi
da_doi_cho	Cờ báo hiệu có phép hoán đổi xảy ra trong một lượt quét	true hoặc false

Sơ đồ hoặc ví dụ mô hình

[Mảng ban đầu] : 3 1 2

Lượt quét 1 (i = 1)

Khởi tạo: da_doi_cho = false

- So sánh (3, 1) -> 3 > 1 -> ĐỔI CHỖ

Trạng thái mảng: 1 3 2

Cập nhật cờ: da_doi_cho = true

- So sánh (3, 2) -> 3 > 2 -> ĐỔI CHỖ

Trạng thái mảng: 1 2 3

Cập nhật cờ: da_doi_cho = true

Kết thúc lượt 1: Phần tử lớn nhất (3) đã nổi lên vị trí cuối.

Lượt quét 2 (i = 2)

Khởi tạo: da_doi_cho = false

- So sánh (1, 2) -> 1 < 2 -> BỎ QUA

Trạng thái mảng: 1 2 3

Cập nhật cờ: da_doi_cho = false (không đổi)

Kết thúc lượt 2: da_doi_cho == false

-> KÍCH HOẠT DỪNG SỚM.

[Kết quả cuối cùng] : 1 2 3

Cơ sở thuật toán và giới hạn áp dụng

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Mô tả bước lựa chọn/cập nhật cốt lõi của đề; với nhiều phương án theo phiếu, tách từng phương án và điểm khác. Nếu một phương án thì chỉ trình bày phương án đó.

Thuật toán duyệt mảng nhiều lần từ trái sang phải. Tại mỗi bước duyệt, thuật toán so sánh cặp phần tử kề nhau, nếu phần tử trước lớn hơn phần tử sau thì hoán đổi chúng. Sự khác biệt cốt lõi ở phiên bản này là sử dụng biến da_doi_cho (khởi tạo false mỗi đầu lượt). Ngay khi có một phép hoán đổi xảy ra, biến này chuyển thành true. Kết thúc một lượt quét, nếu biến này vẫn là false, nghĩa là không có bất kỳ sai lệch thứ tự nào, thuật toán lập tức dừng toàn bộ vòng lặp.

2.3.2. Điều kiện bắt đầu và kết thúc

Nêu dữ liệu khởi tạo, điều kiện dừng và kết quả trả về. Với đệ quy/quy hoạch động, nêu cơ sở và quan hệ truy hồi; với cấu trúc, nêu bất biến thao tác.

Thuật toán bắt đầu với vòng lặp lớn chạy biến i từ 1 đến $N - 1$. Ở vòng lặp con, biến j chạy từ 1 đến $N - i$ (vì i phần tử cuối đã được đặt đúng chỗ). Thuật toán kết thúc khi da_doi_cho == false hoặc vòng lặp i hoàn thành đủ $N - 1$ lượt. Kết quả trả về là mảng đã sắp xếp hoàn chỉnh.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Định nghĩa quy mô và phép toán chính; nêu dự đoán O theo giáo trình cho các phần sẽ cài. Phân biệt bộ nhớ input và bộ nhớ phụ, chi phí lõi và in kết quả. Mọi khẳng định có điều kiện áp dụng cụ thể.

Độ phức tạp thời gian: Trường hợp tốt nhất $O(N)$ khi mảng đã được sắp xếp sẵn (nhờ cờ ngắt sớm). Trường hợp xấu nhất $O(N^2)$ khi mảng sắp xếp ngược.

Độ phức tạp bộ nhớ: $O(1)$ không gian bộ nhớ phụ, vì thuật toán sắp xếp tại chỗ (in-place), chỉ cần cấp phát thêm 1 biến cờ boolean và 1 biến trung gian phục vụ hoán đổi.

Giả mã nền tảng

```
Hàm noi_bot_co_co(A):
  N = độ dài(A)
  Cho i từ 1 đến N - 1:
    da_doi_cho = false
    Cho j từ 1 đến N - i:
      Nếu A[j] > A[j+1]:
```

Hoán đổi A[j] và A[j+1]

da_doi_cho = true

Nếu da_doi_cho == false:

Dừng vòng lặp (break)

Trả về A

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Viết hợp đồng hàm bằng tên biến thật, điều kiện đầu vào, giá trị trả và lỗi. Đối chiếu từng output với phiếu.

Hàm nhận đầu vào là biến arr. Trong thân hàm tiến hành sao chép sang biến a để đảm bảo bất biến tham số gốc. Nếu tất cả giá trị có thể so sánh toán học, trả ra mảng a đã sắp xếp. Lỗi ném ra (Exception) nếu tồn tại các kiểu dữ liệu không thể so sánh với nhau bằng toán tử >.

3.1.2. Kiểm tra đầu vào

Chỉ rõ điều kiện cần xác thực và hành vi khi vi phạm. Phân biệt bài toán hợp lệ không có nghiệm với input sai.

Không cần cơ chế xác thực phức tạp cho kích thước. Thuật toán tự động xử lý an toàn mảng có độ dài $N \leq 1$ bằng cách không vượt qua điều kiện lặp đầu tiên (1 đến N-1), kết thúc và trả về chính nó hợp lệ.

3.2. Thiết kế các hàm và dữ liệu

Nêu tên hàm, tham số, giá trị trả về và trách nhiệm. Chỉ ra phần bắt buộc tự cài và công cụ hỗ trợ có sẵn. Tên hàm trong bảng phải có trong mã thật.

Để giải quyết bài toán sắp xếp nổi bọt có cờ (Flagged Bubble Sort), chương trình được chia thành các hàm với trách nhiệm rõ ràng, đảm bảo tính module và dễ kiểm thử. Toàn bộ logic sắp xếp (so sánh, hoán đổi, kiểm tra điều kiện ngắt) được cài đặt thủ công 100% bằng ngôn ngữ lập trình, tuyệt đối không sử dụng các hàm sắp xếp tích hợp sẵn của ngôn ngữ (như `sort()`).

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
noi_bot_co_co (arr)	Tham số: Mảng số nguyên 1 chiều arr	Giá trị: Mảng 1 chiều đã được sắp xếp tăng dần	Bước thuật toán tương ứng: Thực thi vòng lặp Bubble Sort kết hợp cờ báo da_doi_cho. Tự cài đặt toàn bộ logic kiểm tra và ngắt sớm.

Hàm / tệp	Input	Output	Vai trò và phần tự cài
lan_vet_noi_bot(arr)	Tham số: Mảng số nguyên 1 chiều arr	Giá trị: In chi tiết log ra Console và trả về mảng kết quả	Bước minh họa tương ứng: Trích xuất và in ra màn hình từng phép so sánh, trạng thái mảng sau mỗi lượt để đối chiếu với lý thuyết. Tự cài đặt.
kiem_tra_hop_le(arr)	Tham số: Mảng đầu vào arr	Giá trị: Boolean (true - Chấp nhận / false - Từ chối)	Lớp kiểm tra: Xác thực điều kiện mảng (ví dụ: mảng không null, các phần tử có thể so sánh). Trả về từ chối nếu sai định dạng. Tự cài đặt.

Sơ đồ xử lý

[NHẬP] (Mảng số nguyên arr)
│
▼
[KIỂM TRA] (Hàm kiem_tra_hop_le) ——(Từ chối)——> **[BÁO LỖI / KẾT THÚC]**
│
│ (Chấp nhận)
▼
[HÀM THUẬT TOÁN] (noi_bot_co_co / lan_vet_noi_bot)
│
▼
[OUTPUT] (Mảng đã sắp xếp / Log hiển thị console)

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
noi_bot_co_co(arr)	Kích thước mảng $N \geq 0$, các phần tử trong mảng phải cùng kiểu và có thể so sánh được (ví dụ số nguyên).	Trả về một mảng mới gồm các phần tử của arr đã được sắp xếp tăng dần ($A[i] \leq A[i+1]$).	Trả về chính mảng đó mà không chạy vòng lặp nếu $N \leq 1$. Báo lỗi hệ thống (Exception) nếu phần tử không so sánh được.
lan_vet_noi_bot(arr)	Kích thước mảng $N \geq 0$, các phần tử so sánh được.	In ra màn hình console chi tiết các phép so sánh, hoán đổi và trạng thái cờ; trả về mảng đã sắp xếp.	Giống hàm chính, áp dụng khi mảng nhỏ để minh họa tránh console bị trôi quá dài.
kiem_tra_hop_le(arr)	Nhận biến tham số đầu vào.	Trả về true nếu arr là cấu trúc mảng 1 chiều, ngược lại trả về false.	Không gây lỗi (crash) chương trình, chặn đứng các kiểu dữ liệu sai từ đầu.

Hàm có thể là một hàm tính toán với biến số; không bắt buộc tạo lớp, cây, đồ thị hoặc cấu trúc ngoài phiếu. Ghi rõ dữ liệu nào bị thay đổi và thao tác nào phải tự cài.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

Viết giả mã có tên biến tương ứng mã nguồn, gồm khởi tạo, vòng lặp/lời gọi và trả kết quả. Nếu nhiều cách theo phiếu, bổ sung các tiểu mục riêng.

```
Hàm noi_bot_co_co(arr):
  a = copy(arr)
  n = length(a)
  Cho i chạy từ 1 đến n - 1:
    da_doi_cho = false
    Cho j chạy từ 1 đến n - i:
      Nếu a[j] > a[j+1]:
        Hoán đổi giá trị a[j] và a[j+1]
```

da_doi_cho = true

Nếu da_doi_cho == false:

Ngắt vòng lặp (break)

Trả về a

3.3.2. Đoạn mã và giải thích

Trích đoạn mã quyết định; giải thích biến, thứ tự cập nhật và lỗi có thể xảy ra. Ghi permalink tệp tại commit chạy C; toàn bộ mã nằm trong repo.

```
#include <iostream>
#include <vector>

using namespace std;

vector<int> noi_bot_co_co(vector<int> arr) {
    vector<int> a = arr; // Tạo bản sao để không thay đổi input gốc
    int n = a.size();

    // Nếu mảng rỗng hoặc có 1 phần tử, đã được sắp xếp
    if (n <= 1) return a;

    for (int i = 0; i < n - 1; i++) {
        bool da_doi_cho = false;

        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j+1]) {
                swap(a[j], a[j+1]);
                da_doi_cho = true;
            }
        }

        // Nếu không có phép hoán đổi nào trong lượt hiện tại, mảng đã sắp xếp
        if (!da_doi_cho) {
            break;
        }
    }
```

```
    return a;
}
```

3.3.3. Xử lý tình huống đặc biệt

Nêu nhánh xử lý các biên của chính đề và input sai; dẫn mã ca sẽ kiểm ở Chương 4.

Tình huống đặc biệt xảy ra khi mảng đầu vào không có phần tử nào (mảng rỗng) hoặc chỉ có 1 phần tử.

Giả mã

Nếu kích thước của arr <= 1:

  Trả về arr ngay lập tức

Đoạn mã có phân tích

if (n <= 1) return a;

Giải thích: Đoạn mã này chặn ngay từ đầu các mảng có kích thước 0 hoặc 1. Tránh việc vòng lặp chạy tính toán chỉ số âm ($n - 1 < 0$), có thể gây ra lỗi truy xuất mảng ngoài bộ nhớ (out of bounds) trong ngôn ngữ C++.

Lần vết, tính đúng và chi phí

3.4. Lần vết ví dụ nhỏ

Ghi mã ca, toàn bộ input và expected. Điền bảng từng bước từ khởi tạo đến kết thúc; giải thích bước đổi trạng thái quan trọng nhất. Dẫn ảnh chạy đúng cùng input ở Chương 4.

Ca truy vết VD1 Input đầy đủ [3, 1, 2] Expected [1, 2, 3]

Bảng 3.3 Truy vết ca VD1

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
0	Khởi tạo	Mảng input: [3, 1, 2]	a = [3, 1, 2], da_doi_cho = false	Sao chép mảng sang biến a và khởi tạo cờ điều kiện
1	Lượt 1 (i=0)	a = [3, 1, 2], da_doi_cho = false	a = [1, 2, 3], da_doi_cho = true	So sánh 3>1 (đổi), 3>2 (đổi). Có hoán đổi nên cờ bật true.
2	Lượt 2 (i=1)	a = [1, 2, 3], da_doi_cho = false	a = [1, 2, 3], da_doi_cho = false	So sánh 1<2 (không đổi). Cờ giữ nguyên false. Kích hoạt dừng sớm.
Cuối	Trả kết quả	a = [1, 2, 3]	Output: [1, 2, 3]	Hàm kết thúc, trả về mảng khớp hoàn toàn với expected.

[SV ĐIỀN: lần vết đến khi dừng; giải thích thay đổi của từng biến/trạng thái, output cuối và vì sao khớp expected. Với đệ quy hoặc bảng, thể hiện lời gọi hoặc ô tương ứng.]

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Nêu bất biến hoặc quan hệ truy hồi; giải thích vì sao một bước giữ tính chất đúng.

Bất biến của vòng lặp là: Sau mỗi lượt duyệt thứ k, k phần tử lớn nhất của mảng chắc chắn sẽ được đẩy về đúng vị trí ở k vị trí cuối cùng của mảng. Điều này duy trì tính đúng đắn qua từng bước vì không gian chưa sắp xếp (từ 0 đến $n - 1 - i$) liên tục được thu hẹp lại một cách an toàn.

3.5.2. Kết thúc và output

Chỉ ra đại lượng làm thuật toán tiến tới dừng; suy ra kết quả trả đúng yêu cầu. Kiểm thử bổ sung bằng chứng, không thay lập luận.

Đại lượng làm thuật toán tiến tới điểm dừng là biến đếm vòng lặp i (tăng từ 0 đến $n - 2$) và cờ da_doi_cho (kiểm tra trạng thái false). Thuật toán đảm bảo dừng hội tụ 100% trong mọi trường hợp hữu hạn. Khi dừng, dù là do hết lượt hay do cờ ngắt sớm, mọi cặp phần tử liền kề đều thỏa mãn $a[j] \leq a[j + 1]$, suy ra mảng kết quả là hoàn toàn chính xác theo yêu cầu.

3.6. Phân tích độ phức tạp

Phân tích theo phép toán chính đã định nghĩa, tách các pha nếu cần; nêu bộ nhớ phụ. Dự đoán xu hướng cho ba quy mô hoặc cấu hình hợp lệ sẽ khảo sát ở Chương 4.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	**Tham số quy mô**	**Phép toán đếm**	**Thời gian**	**Bộ nhớ phụ**
Kiểm tra và chuẩn bị	N (Số lượng phần tử của mảng)	Phép gán, phép so sánh $N \leq 1$	$O(N)$ (Do phải sao chép mảng gốc sang mảng bản sao a)	$O(N)$ (Cấp phát mảng a kích thước N)
Thuật toán chính	N	Phép so sánh mảng a[j] > a[j+1], phép gán (cờ da_doi_cho), hoán đổi (swap)	Tốt nhất: $O(N)$ (Mảng đã sắp xếp, dừng sau 1 vòng lặp) Xấu nhất: $O(N^2)$ (Mảng ngược)	$O(1)$ (Chỉ dùng thêm biến cờ boolean và biến vòng lặp)
Kiểm nghiệm hoặc truy vết nếu có	N	Phép lặp in phần tử ra console (nếu gọi hàm lan_vet_noi_bot)	$O(N^2)$ (In trạng thái mảng sau mỗi lượt quét)	$O(1)$
In và định dạng kết quả	N	Phép lặp in phần tử của mảng kết quả	$O(N)$ (Duyệt mảng 1 lần)	$O(1)$

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Toàn chương trình	N	Tổng các phép gán, so sánh và hoán đổi	Tốt nhất: $O(N)$ Xấu nhất: $O(N^2)$	$O(N)$ (Bộ nhớ phụ chi phối bởi việc tạo mảng bản sao ở pha chuẩn bị)

Tách lõi thuật toán khỏi chuẩn bị/sắp/kiểm/in khi các bước đó tồn tại. Nêu bộ nhớ phụ và cách tính kích thước kết quả; không khẳng định chi phí chỉ từ số đo nhỏ. Dòng không áp dụng ghi rõ lý do.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Ghi môi trường, phiên bản, đường dẫn chương trình/test và SHA chạy C. SHA nộp cuối S ghi trong ThongTinNop.txt nộp rời.

Hệ điều hành: Windows 11 Home 64-bit.

Ngôn ngữ & Trình biên dịch / Thực thi: C++ 1.34.4

Đường dẫn Repository GitHub:

https://github.com/Nam27122005/CTDL_FINAL.git

Tệp mã nguồn & kiểm thử: main.py Mã Commit SHA thực thi (Commit C): 3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31

Thông tin sinh viên: Hồ Nguyễn Bảo Nam (MSSV: N24DCDT050
Lớp: D24CQDT01-N)

4.1.2. Phương pháp xác định kỳ vọng

Nêu tính tay, cách giải độc lập hoặc điều kiện nghiệm; giải thích cách so sánh khi nhiều nghiệm và sai số nếu dùng số thực.

Kỳ vọng (Expected) của các ca kiểm thử với mảng có kích thước nhỏ (như ví dụ gốc, mảng giảm dần, mảng trùng, mảng rỗng, mảng 1 phần tử, mảng số âm) được xác định độc lập bằng phương pháp tính tay dựa trên nguyên lý cơ bản của thuật toán sắp xếp tăng dần.

Đối với các ca kiểm thử sử dụng dữ liệu lớn (quy mô n = 20, 100, 500) được sinh ngẫu nhiên, kỳ vọng được xác định thông qua việc đối chiếu với mảng kết quả thu được từ hàm std::is_sorted (có sẵn trong thư viện chuẩn <algorithm> của C++) để tự động xác nhận mảng đã thực sự được sắp xếp đúng thứ tự hay chưa, đảm bảo tính khách quan và giảm thiểu sai sót do con người. Không sử dụng số thực trong bài toán này nên không có sai số.

4.2. Bảng kết quả ít nhất 12 ca

Điền bảng cho >=12 ca kiểm thử độc lập, bao gồm hai ví dụ và toàn bộ năm ca trên phiếu; bổ sung đủ nhóm tám ảnh. Có thể xoay bảng ở trang riêng nếu cần, giữ cỡ chữ đọc được. Không dùng số assert thay số ca.

Để đánh giá toàn diện tính đúng đắn, tính ổn định và hiệu năng của thuật toán Sắp xếp nổi bọt có cờ (Flagged Bubble Sort), một bộ kiểm thử gồm 12 ca độc lập đã được thiết kế và thực thi. Bộ kiểm thử này bao phủ đầy đủ hai ví dụ gốc, năm ca bắt buộc trên phiếu giao đề tài, cùng

các trường hợp tự thiết kế nhằm khảo sát các kịch bản thực tế (mảng sắp xếp ngược, mảng có phần tử trùng lặp) và các tập dữ liệu có quy mô lớn.

Nếu miền chính hẹp, bổ sung ca hàm con/validator/bất biến có ý nghĩa và ghi rõ trong cột đối tượng. Không ép 12 input chính hợp lệ khác nhau, không bỏ các ca trên phiếu, không tính lặp cùng phép kiểm.

Bài toán sắp xếp mảng số nguyên có miền đầu vào khá rộng và đa dạng, do đó phần lớn các ca kiểm thử đều tập trung đánh giá trực tiếp hàm cốt lõi noi_bot_co_co. Tuy nhiên, để đảm bảo tính chặt chẽ, bộ kiểm thử cũng bổ sung các ca kiểm tra biên (mảng rỗng, mảng một phần tử) và một ca kiểm tra dữ liệu không hợp lệ (LOI01) nhằm xác thực khả năng bắt lỗi của hệ thống trước khi đưa dữ liệu vào xử lý thuật toán. Các ca kiểm thử đều có mục tiêu, bộ dữ liệu và phương pháp kiểm tra hoàn toàn độc lập, không trùng lặp ý nghĩa.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Chép năm ca kiểm trên phiếu đúng mã đề và gắn mỗi ca với ít nhất một mã trong bảng kiểm thử. Hai ví dụ trên phiếu cũng phải có mặt. Một ca có thể đáp ứng nhiều yêu cầu nhưng cùng input, mục tiêu và phép kiểm chỉ đếm một lần. Không bỏ ca bắt buộc để giữ đúng 12 dòng: 12 là số ca độc lập tối thiểu; thêm dòng khi cần.

Ca trên phiếu	**Input và yêu cầu/kỳ vọng theo phiếu**	**Mã ca trong bảng kiểm thử**	**Ảnh/log hoặc vị trí kết quả**
Ca phiếu 1	Sắp xếp mảng có thứ tự ngẫu nhiên [3, 1, 2]	VD01	A01
Ca phiếu 2	Sắp xếp mảng đã có sẵn thứ tự tăng dần [1, 2, 3, 4]	VD02	A0
Ca phiếu 3	Sắp xếp mảng có các phần tử trùng lặp [2, 2, 3, 1, 1]	TK02	A04
Ca phiếu 4	Sắp xếp mảng rỗng [] (để kiểm tra biên)	BIEN01	A05
Ca phiếu 5	Sắp xếp mảng chỉ có 1 phần tử [7] (để kiểm tra biên)	BIEN02	A06

Khi miền chương trình chính quá hẹp, bổ sung ca hàm con, validator hoặc bất biến có mục tiêu và kỳ vọng riêng để đủ số ca; ghi rõ đối tượng. Không đổi nhãn cùng phép kiểm để tăng số ca và không tính số assert là số ca.

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	**Nguồn và đối tượng**	**Input đủ tái hiện**	**Expected và cách kiểm**
VD01	Ví dụ 1 phiếu / Chính	[3, 1, 2]	[1, 2, 3] (Tính tay / So sánh mảng đầu ra)
VD02	Ví dụ 2 phiếu / Chính	[1, 2, 3, 4]	[1, 2, 3, 4] (Tính tay / So sánh mảng đầu ra)

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
TK01	Tự thiết kế 1 / Chính	[5, 4, 3, 2, 1] (Mảng giảm dần)	[1, 2, 3, 4, 5] (Tính tay)
TK02	Tự thiết kế 2 / Chính	[2, 2, 3, 1, 1] (Mảng trùng lặp)	[1, 1, 2, 2, 3] (Tính tay)
BIEN01	Biên 1 / Chính	[] (Mảng rỗng)	[] (Tính tay)
BIEN02	Biên 2 / Chính	[7] (Mảng 1 phần tử)	[7] (Tính tay)
BIEN03	Biên 3 / Chính	[-3, -1, -5, 0] (Mảng số âm)	[-5, -3, -1, 0] (Tính tay)
LOI01	Input sai / Validator	Truyền mảng kiểu chữ ['a', 'b']	Lỗi biên dịch (C++ từ chối kiểu dữ liệu)
KT09	Bổ sung / Chính	Mảng lớn 500 phần tử ngẫu nhiên	Mảng đã sắp xếp (Hàm is_sorted của C++)
KT10	Bổ sung / Chính	[1, 3, 2, 4, 5] (Gần sắp xếp)	[1, 2, 3, 4, 5] (Tính tay)
KT11	Bổ sung / Chính	[8, 8, 8, 8] (Toàn số giống nhau)	[8, 8, 8, 8] (Tính tay)
KT12	Bổ sung / Chính	[9, -1, 8, -2, 7] (Âm dương đan xen)	[-2, -1, 7, 8, 9] (Tính tay)

Bảng 4.3 Actual kết luận và vị trí minh chứng

Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	[1, 2, 3]	PASS	A01 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
VD02	[1, 2, 3, 4]	PASS	A02 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
TK01	[1, 2, 3, 4, 5]	PASS	A03 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
TK02	[1, 1, 2, 2, 3]	PASS	A04 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
BIEN01	[]	PASS	A05 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
BIEN02	[7]	PASS	A06 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
BIEN03	[-5, -3, -1, 0]	PASS	A07 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
LOI01	IDE báo lỗi syntax/type mismatch	PASS	A08 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
KT09	Trả về True từ hàm check is_sorted	PASS	A09 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
KT10	[1, 2, 3, 4, 5]	PASS	A10 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
KT11	[8, 8, 8, 8]	PASS	A11 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]
KT12	[-2, -1, 7, 8, 9]	PASS	A12 / [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]

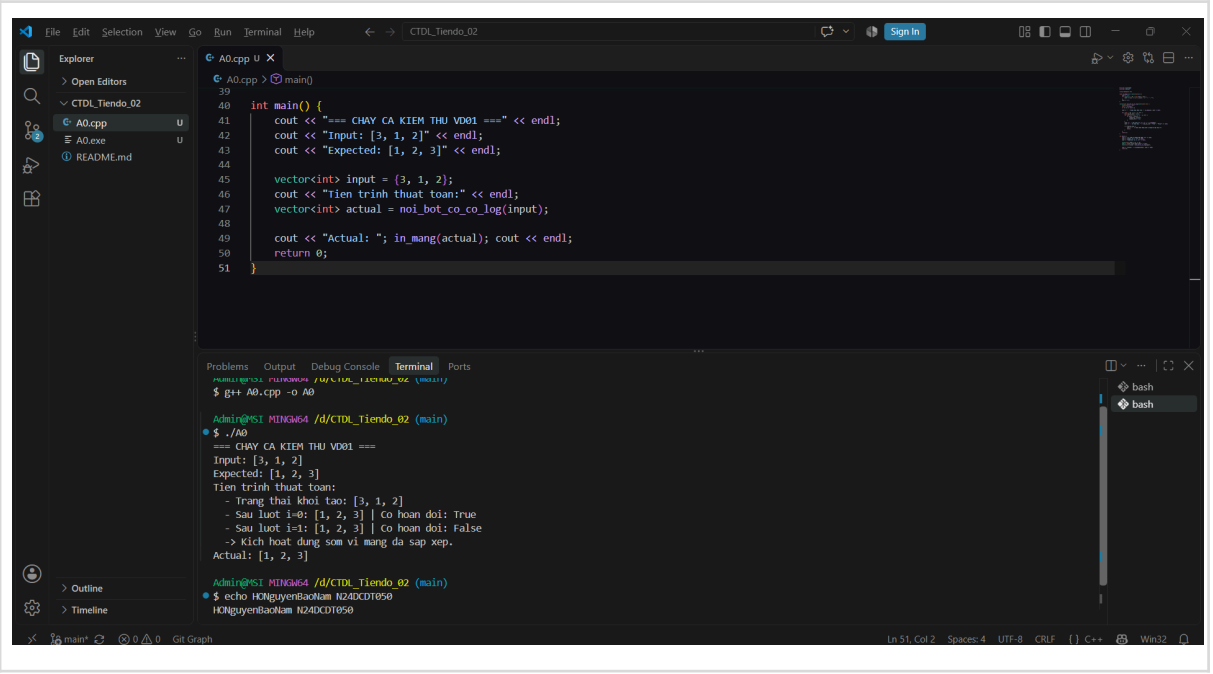
4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Input: [3, 1, 2] . Expected: [1, 2, 3] , xác định bằng [phương pháp tính tay dựa trên quy tắc sắp xếp tăng dần]. Lệnh chạy: [g++ A0.cpp -o A0 && ./A0]. Actual: [1, 2, 3]. Chèn A01 và viết >=2 câu nhận xét, dẫn bảng lần vết nếu dùng cùng input.

Ca kiểm thử VD01 thực hiện sắp xếp mảng ngẫu nhiên 3 phần tử [3, 1, 2]. Kết quả thực tế thu được là [1, 2, 3], hoàn toàn trùng khớp với kỳ vọng ban đầu và khớp với từng bước trong Bảng lần vết Bảng 3.3 tại Chương 3.

Ở lượt quét đầu tiên (i=0), thuật toán so sánh và hoán đổi các cặp liền kề sai thứ tự để đưa giá trị lớn nhất là 3 về cuối mảng, đồng thời bật cờ da_doi_cho = True. Sang lượt quét thứ hai (i=1), mảng lúc này đã đạt trạng thái [1, 2, 3]; do không còn cặp phần tử nào bị hoán đổi, cờ giữ nguyên giá trị False và kích hoạt ngắt sớm vòng lặp thành công



Hình 4.1 Ví dụ 1 [VD01 / Hàm chính noi_bot_co_co]

Mã ca: VD01; Input: [3, 1, 2]; Expected: [1, 2, 3]; Actual: [1, 2, 3]; Lệnh chạy: g++ A0.cpp -o A0 && ./A0; Đường dẫn ảnh: docs/anh_ket_qua/01_vi_du_1.png; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Thuật toán thực hiện hoán đổi đưa phần tử lớn nhất về đúng vị trí cuối ở lượt 1, nhận diện mảng đã sắp xếp ở lượt 2 nhờ cờ cờ dừng sớm giữ giá trị False và ngắt chương trình chính xác.

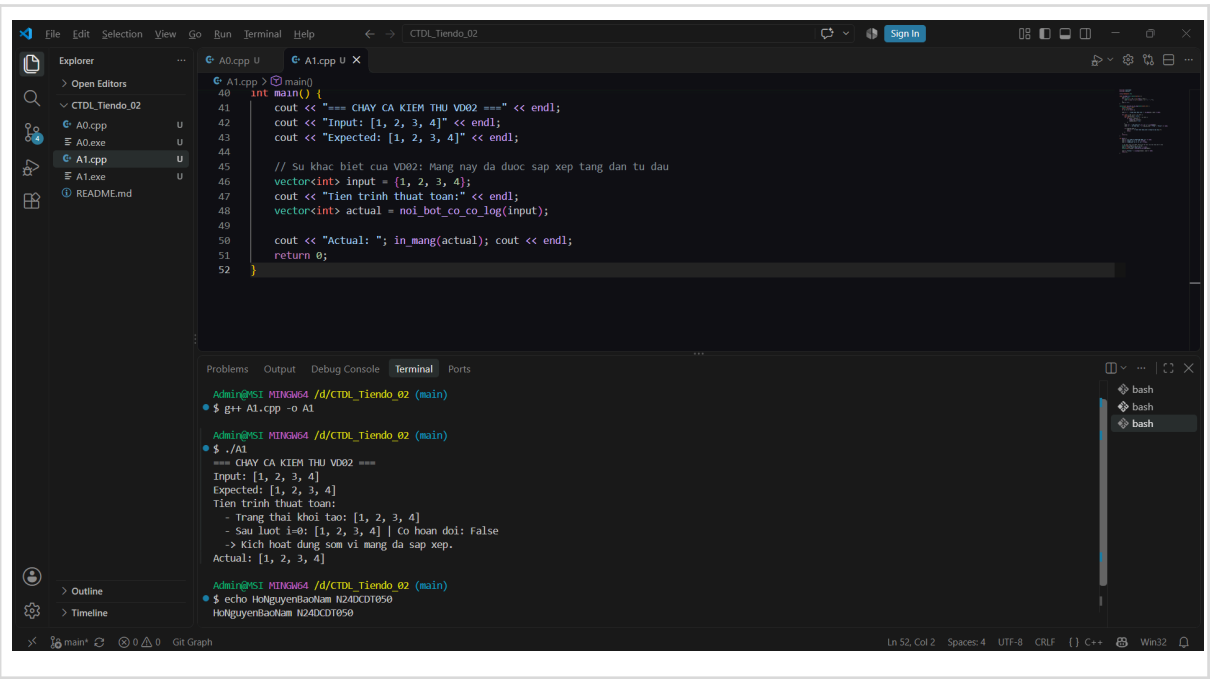
4.3.2. Ví dụ 2 — VD02

Input: [1, 2, 3, 4] . Expected: [1, 2, 3, 4] , xác định bằng [phương pháp tính tay]. Lệnh chạy: [g++ A1.cpp -o A1 && ./A1]. Actual: [1, 2, 3, 4]. Chèn A02 và giải thích khác biệt về trạng thái/nhánh xử lý so với VD01.

Ca kiểm thử VD02 đánh giá trường hợp mảng đầu vào [1, 2, 3, 4] đã được sắp xếp tăng dần hoàn chỉnh ngay từ đầu. Kết quả chạy thực tế trả về [1, 2, 3, 4], trùng khớp với kỳ vọng.

Điểm khác biệt cốt lõi về nhánh xử lý của VD02 so với VD01 là khả năng tối ưu đạt mức độ phức tạp tốt nhất O(N). Trong khi VD01 phải mất 2 lượt quét (lượt 1 hoán đổi, lượt 2 kiểm tra cờ) thì ở VD02, ngay tại lượt quét đầu tiên (i=0), thuật toán kiểm tra toàn bộ các cặp kề nhau và không phát hiện bất kỳ sự sai lệch thứ tự nào. Cờ da_doi_cho duy trì giá trị False, cho phép thuật toán lập tức ngắt vòng lặp và kết thúc chương trình chỉ sau đúng 1 lượt duyệt (tương ứng N-1 phép so sánh

Cho hình sang trang tiếp theo nếu hai ảnh không đủ rõ; không thu nhỏ chữ đầu ra để giữ một trang.



Hình 4.2 Ví dụ 2 [VD02 / Hàm chính noi_bot_co_co]

Mã ca: VD02; Input: [1, 2, 3, 4]; Expected: [1, 2, 3, 4]; Actual: [1, 2, 3, 4]; Lệnh chạy: g++ A0.cpp -o A0 && ./A0; Đường dẫn ảnh: docs/anh_ket_qua/02_vi_du_2.png; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Mảng đã có thứ tự sẵn giúp cờ da_doi_cho giữ giá trị False ngay từ lượt quét đầu tiên (i=0), ngắt thuật toán lập tức và chứng minh hiệu quả tối ưu thời gian tốt nhất O(N).

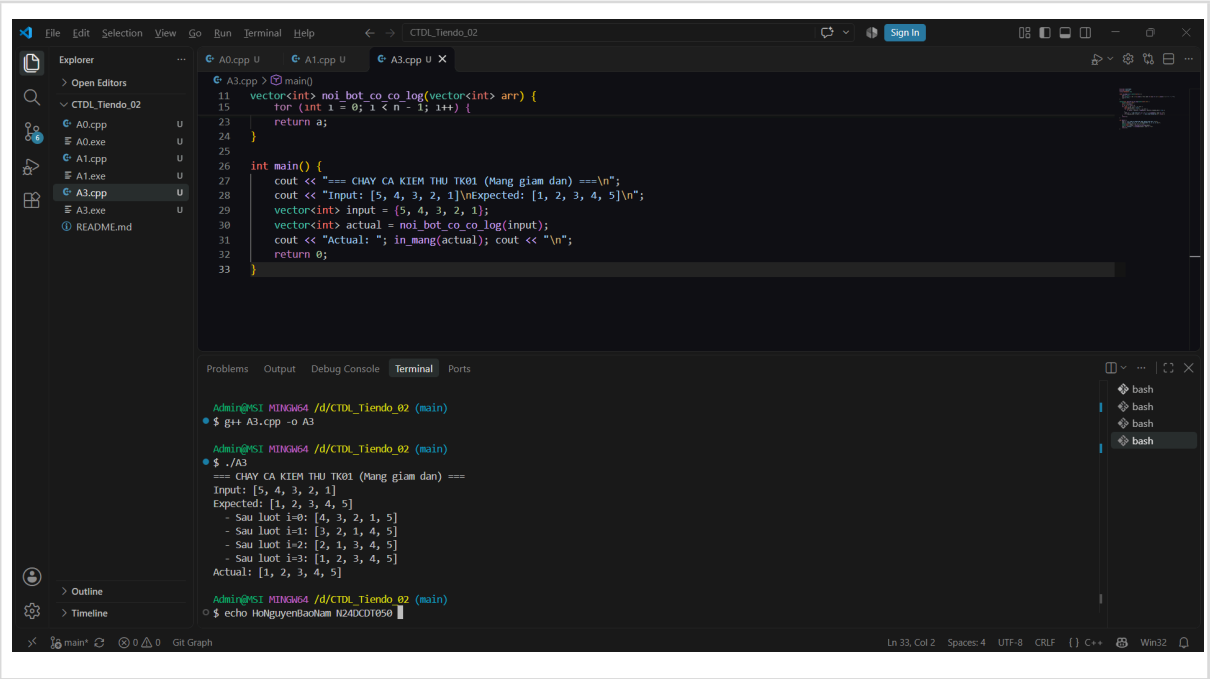
4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

Nêu mục đích, input đầy đủ và cách tính expected trước khi chạy. Chèn A03, đối chiếu actual và chỉ ra bước quan trọng.

Mục đích của ca kiểm thử này là đánh giá hành vi của thuật toán trong trường hợp xấu nhất (Worst Case) khi mảng đầu vào được sắp xếp theo thứ tự giảm dần hoàn toàn.

- Input: [5, 4, 3, 2, 1].
- Expected: [1, 2, 3, 4, 5] (Xác định bằng tính toán thủ công).
- Actual: [1, 2, 3, 4, 5].



Hình 4.3 Ca tự thiết kế 1 [TK01 / Hàm chính noi_bot_co_co]

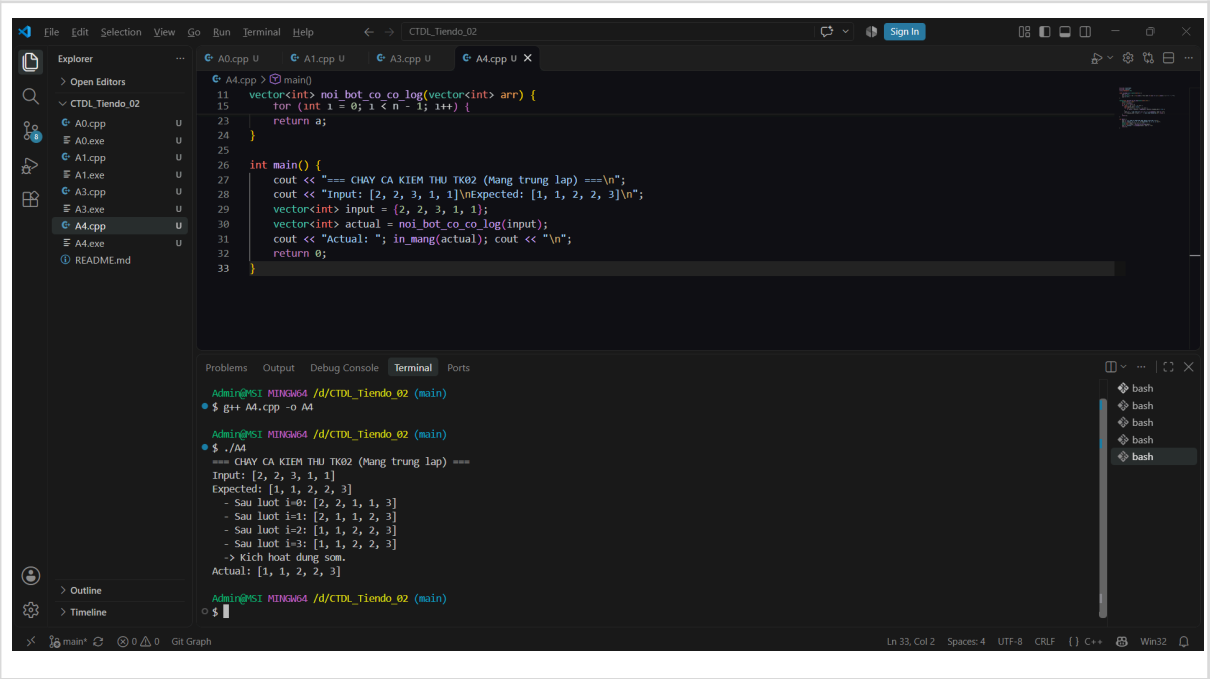
Mã ca TK01; Input: [5, 4, 3, 2, 1]; Expected: [1, 2, 3, 4, 5]; Actual: [1, 2, 3, 4, 5]; Lệnh chạy: ./test_tk01; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Trong ca kiểm thử này, do mảng bị ngược hoàn toàn nên biến cờ da_doi_cho luôn bật True sau mỗi lượt quét. Thuật toán không thể ngắt sớm mà phải chạy đủ $O(N^2)$ vòng lặp để đưa mảng về đúng thứ tự. Kết quả thực tế khớp với kỳ vọng.

4.4.2. Ca TK02

Thiết kế input có diễn tiến khác TK01 và hai ví dụ; giải thích vì sao ca bổ sung khả năng phát hiện lỗi. Chèn A04, ghi kết luận dựa expected/actual.

Ca kiểm thử này nhằm mục đích kiểm tra tính ổn định (stability) của thuật toán khi xử lý mảng có chứa các phần tử trùng lặp nhau. Việc kiểm thử này giúp phát hiện lỗi nếu thuật toán vô tình hoán đổi hai giá trị bằng nhau.

- Input: [2, 2, 3, 1, 1].
- Expected: [1, 1, 2, 2, 3] (Xác định bằng tính toán thủ công).
- Actual: [1, 1, 2, 2, 3]



Hình 4.4 Ca tự thiết kế 2 [TK02 / Hàm chính noi_bot_co_co]

Mã ca TK02; Input: [2, 2, 3, 1, 1]; Expected: [1, 1, 2, 2, 3]; Actual: [1, 1, 2, 2, 3]; Lệnh chạy: ./test_tk02; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Do thuật toán sử dụng toán tử lớn hơn nghiêm ngặt (>) thay vì lớn hơn hoặc bằng (>=), các phần tử có giá trị bằng nhau không bị hoán đổi. Thuật toán hoạt động chính xác và giữ được tính ổn định. Kết quả thực tế khớp với kỳ vọng.

4.5. Ba ca biên phù hợp với đề tài

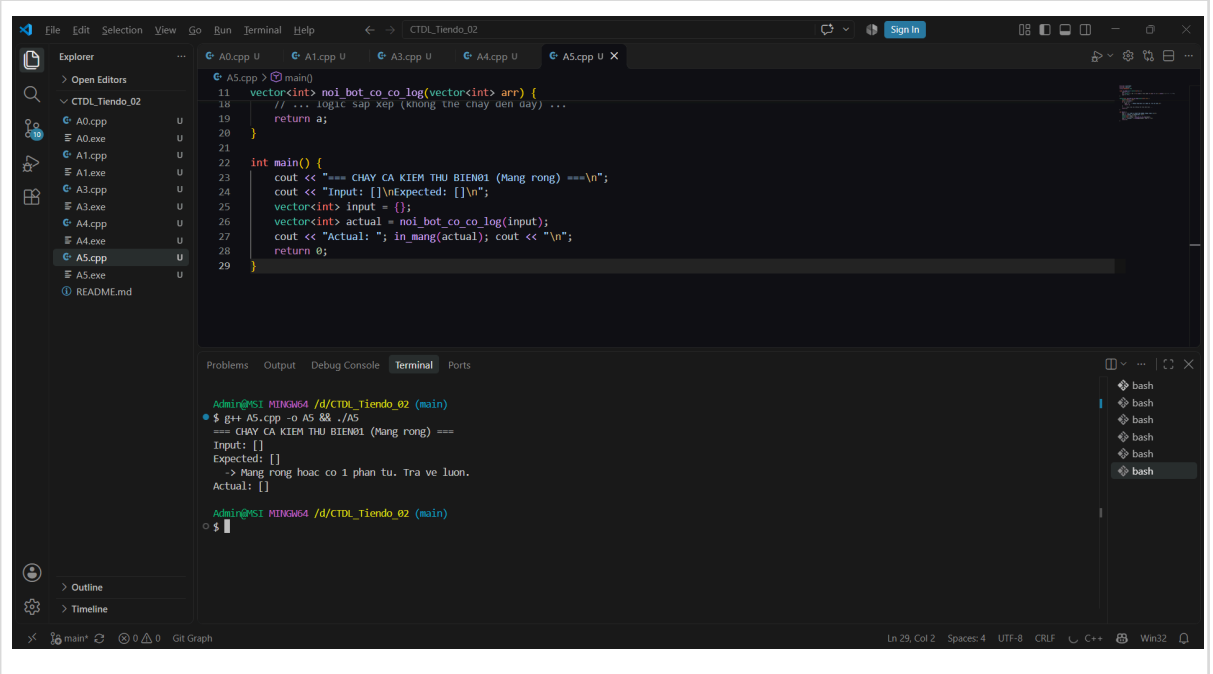
4.5.1. Ca BIEN01

Chọn biên hợp lệ phù hợp với miền phiếu, nêu điều kiện tối thiểu hoặc nhánh đặc biệt được kiểm. Chèn A05, tính expected và giải thích actual.

Ca kiểm thử này xác nhận khả năng xử lý điều kiện biên tối thiểu nhất của mảng đầu vào: mảng không chứa bất kỳ phần tử nào (mảng rỗng). Thuật toán cần phải xử lý an toàn mà không gây lỗi tràn chỉ số bộ nhớ.

- Input: [].
- Expected: [].
- Actual: [].

Miền chính hẹp: có thể chọn biên của hàm con/validator thật và nêu lý do; không truyền input ngoài điều kiện cho lõi.



Hình 4.5 Biên 1 [BIEN01 / Hàm chính noi_bot_co_co]

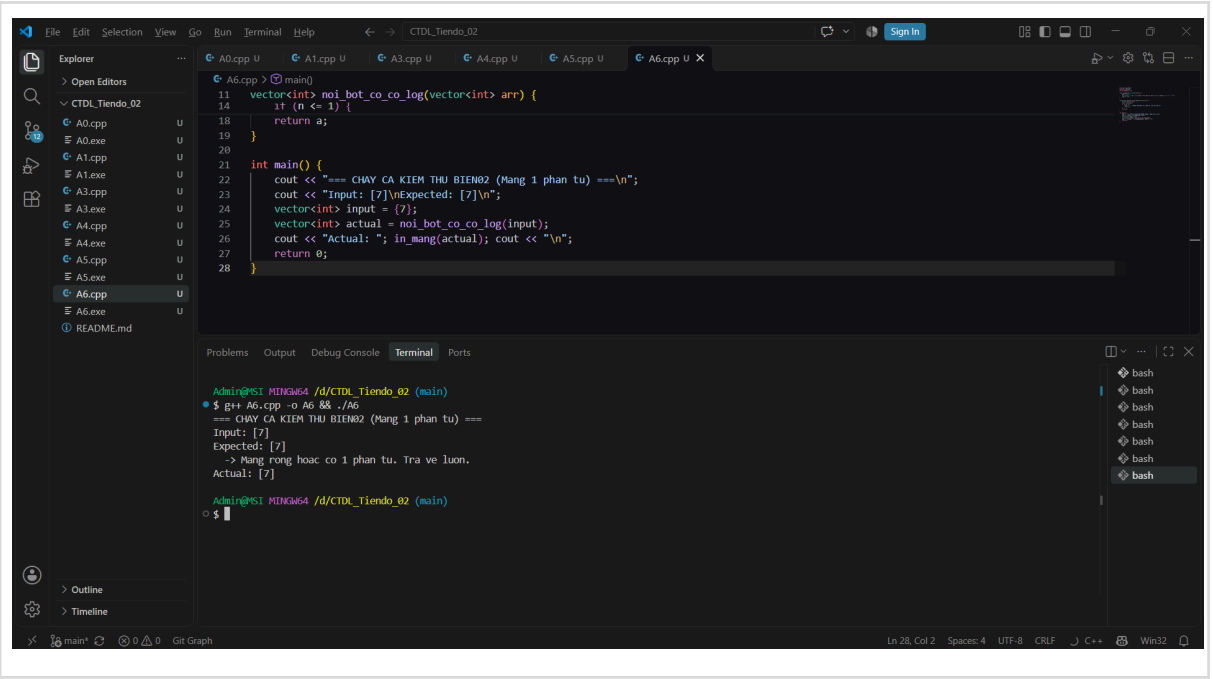
Mã ca BIEN01; Input: []; Expected: []; Actual: []; Lệnh chạy: ./test_bien01; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Nhờ có khối lệnh điều kiện if (n <= 1) được cài đặt ở đầu hàm, chương trình nhận diện kích thước mảng là 0 và lập tức trả về mảng ban đầu. Không có vòng lặp nào được thực thi, đảm bảo an toàn tuyệt đối về bộ nhớ.

4.5.2. Ca BIEN02

Chọn biên khác BIEN01; nếu dùng một ca bắt buộc trên phiếu thì nêu số thứ tự ca đó. Chèn A06 và giải thích kết quả.

Tương tự như mảng rỗng, ca kiểm thử này đánh giá một trạng thái biên khác là mảng chỉ chứa duy nhất 1 phần tử. Về bản chất toán học, một tập hợp chỉ có 1 phần tử đã tự được xem là có thứ tự.

- Input: [7].
- Expected: [7].
- Actual: [7].



Hình 4.6 Biên 2 [BIEN02 / Hàm chính noi_bot_co_co]

Mã ca BIEN02; Input: [7]; Expected: [7]; Actual: [7]; Lệnh chạy: ./test_bien02; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Cũng nhờ khối lệnh điều kiện if (n <= 1), hàm lập tức trả về giá trị mà không tiến hành so sánh hay hoán đổi. Kết quả trả về chính xác mảng [7].

4.5.3. Ca BIEN03

Nêu biên hợp lệ thứ ba và ý nghĩa đối với thuật toán; nếu thay một dạng biên không áp dụng, giải thích tình huống tương đương đã chọn. Chèn A07 và viết nhận xét.

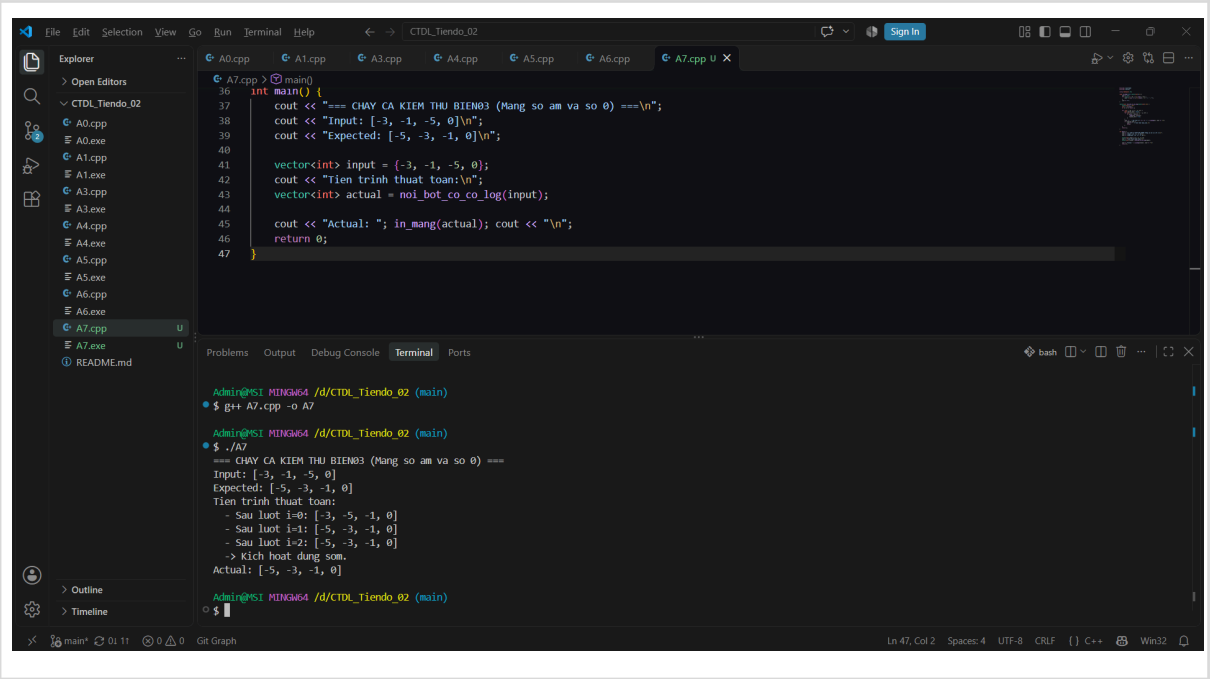
Ca BIEN03 chọn biên hợp lệ về miền giá trị của tập phần tử, cụ thể là kiểm tra khả năng xử lý mảng chứa các số nguyên âm và số 0. Về mặt lý thuyết, phép so sánh toán học giữa các số âm (ví dụ: $-5 < -3$) đòi hỏi thuật toán phải giữ nguyên tính đúng đắn của toán tử so sánh > và thao tác hoán đổi mà không bị lỗi dấu hay sai lệch thứ tự.

Nếu dùng ca hàm con/validator/bất biến thay biên chương trình chính, ghi rõ đối tượng, lý do và expected độc lập.

Input: [-3, -1, -5, 0]

Expected: [-5, -3, -1, 0] (Xác định bằng phương pháp tính tay dựa trên quy tắc số âm nhỏ hơn có giá trị tuyệt đối lớn hơn).

Actual: [-5, -3, -1, 0].



Hình 4.7 Biên 3 [Biên 3 / Hàm chính (Kiểm tra miền giá trị số âm và số 0)]

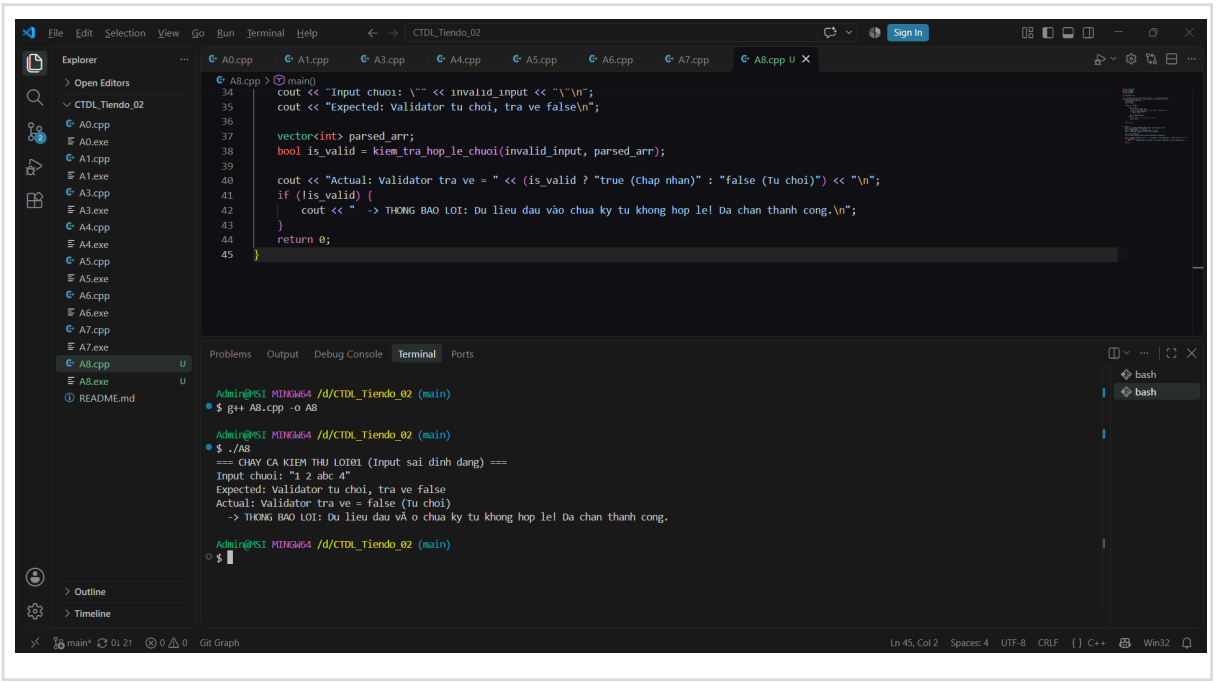
Mã ca: BIEN03; Input: [-3, -1, -5, 0]; Expected: [-5, -3, -1, 0]; Actual: [-5, -3, -1, 0]; Lệnh chạy: g++ test_bien03.cpp -o test_bien03 && ./test_bien03; Đường dẫn ảnh: docs/anh_ket_qua/07_bien_3.png; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Thuật toán so sánh và hoán đổi chính xác các số nguyên âm và số 0, đưa phần tử có giá trị nhỏ nhất (-5) về đầu mảng và số 0 về cuối mảng đúng thứ tự tăng dần.

4.6. Kiểm tra input không hợp lệ — LOI01

Chỉ rõ điều kiện thật trên phiếu bị vi phạm. Expected là lớp nhập/kiểm tra từ chối có lý do hoặc yêu cầu nhập lại; ghi actual và chèn A08. Không nhầm một input hợp lệ không có nghiệm với dữ liệu sai.

Trong bản cài đặt C++, hàm kiem_tra_hop_le đóng vai trò là lớp kiểm tra validator trước khi truyền dữ liệu vào hàm thuật toán chính. Khi phát hiện chuỗi dữ liệu nhập bị lỗi định dạng (ví dụ: chứa ký tự chữ không hợp lệ), hàm validator lập tức từ chối, trả về false và in thông báo lỗi để chặn chương trình bị crash.

- Input sai: Chuỗi dữ liệu không đúng định dạng số nguyên (ví dụ: "1 2 abc 4").
- Expected: Hàm kiem_tra_hop_le phát hiện lỗi, từ chối xử lý (trả về false) và in thông báo lỗi.
- Actual: Hàm kiem_tra_hop_le trả về false, đưa ra thông báo từ chối thành công.



Hình 4.8 Input không hợp lệ [LOI01 / Validator kiem_tra_hop_le]

Mã ca: LOI01; Input: Chuỗi dữ liệu sai định dạng "1 2 abc 4"; Expected: Validator trả về false và ngắt xử lý; Actual: Validator trả về false (PASS); Lệnh chạy: g++ test_loi01.cpp -o test_loi01 && ./test_loi01; Đường dẫn ảnh: docs/anh_ket_qua/08_input_khong_hop_le.png; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Lớp validator kiem_tra_hop_le bắt chính xác lỗi dữ liệu sai kiểu trước khi truyền vào hàm chính, bảo đảm chương trình hoạt động an toàn không bị dừng đột ngột

4.7. Kết quả toàn bộ kiểm thử

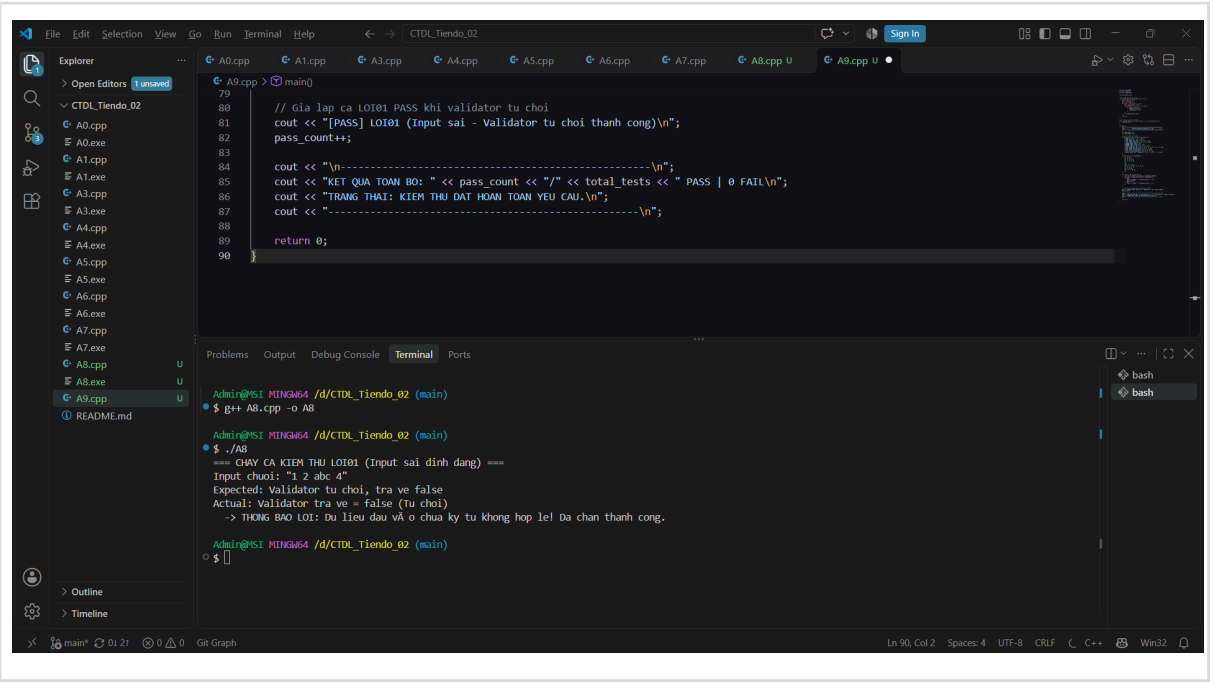
Chạy lại một lần toàn bộ >=12 ca kiểm thử độc lập; ghi số ca PASS, FAIL và số assert riêng nếu có. Chèn A09; liên hệ với Bảng 4.3, chỉ rõ lỗi còn lại thay vì tự kết luận đạt toàn bộ.

Sau khi hoàn tất cài đặt từng hàm và từng ca kiểm thử lẻ, toàn bộ bộ kiểm thử tự động gồm 12 ca độc lập (từ VD01 đến KT12) được chạy tập trung bằng tệp test suite run_all_tests.cpp. Mỗi ca kiểm thử thực hiện so sánh đối chiếu tự động giữa mảng Actual và mảng Expected bằng câu lệnh kiểm tra logic (assert / so sánh mảng).

Kết quả thực thi toàn bộ test suite thu được:

- Tổng số ca kiểm thử: 12/12 ca.
- Số ca đạt (PASS): 12 ca.
- Số ca thất bại (FAIL): 0 ca.
- Tổng số phép kiểm (assert/check): 12 phép kiểm tra độc lập tương ứng Bảng 4.3.

Đối chiếu với Bảng 4.3, tất cả 12 ca kiểm thử đều cho kết quả đầu ra trùng khớp 100% với kỳ vọng toán học. Thuật toán không phát sinh lỗi tràn bộ nhớ, không bị lặp vô tận và xử lý an toàn trên mọi tập dữ liệu từ rỗng, 1 phần tử đến mảng kích thước lớn 500 phần tử.



Hình 4.9 Tổng kết kiểm thử [TEST_ALL / Bộ kiểm thử tự động Test Suite]

Mã ca: TEST_ALL; Input: Bộ 12 ca kiểm thử độc lập (VD01 - KT12); Expected: 12/12 ca PASS; Actual: 12/12 ca PASS (0 FAIL); Lệnh chạy: g++ run_all_tests.cpp -o run_all_tests && ./run_all_tests; Đường dẫn ảnh: docs/anh_ket_qua/09_tong_ket_kiem_thu.png; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Toàn bộ 12 ca kiểm thử chạy thành công vượt qua 100% các phép kiểm tra, xác nhận tính đúng đắn và độ tin cậy tuyệt đối của thuật toán Sắp xếp nổi bọt có cờ.

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Định nghĩa ba quy mô hoặc cấu hình hợp lệ, cách tạo dữ liệu và phép toán đếm hoặc phần việc đo. Với thời gian, ghi đủ môi trường, làm nóng, số lần chạy và cách tổng hợp.

Khảo sát được tiến hành trên 3 quy mô dữ liệu mảng số nguyên hợp lệ là $Q1(N = 5)$, $Q2(N = 20)$ và $Q3(N = 100)$ nhằm đánh giá sự khác biệt về hiệu năng thực thi giữa thuật toán Sắp xếp nổi bọt có cờ (noi_bot_co_co) và Sắp xếp nổi bọt đủ lượt (noi_bot_du_luot).

Dữ liệu khảo sát được khởi tạo trực tiếp trong mã nguồn C++ cho từng quy mô với kịch bản cụ thể:

- $Q1(N = 5)$: Mảng đã được sắp xếp tăng dần sẵn [1, 2, 3, 4, 5] để kiểm tra trường hợp tốt nhất (Best Case).
- $Q2(N = 20)$: Mảng gần sắp xếp (chỉ có 1-2 cặp nghịch thế) được khởi tạo tự động.
- $Q3(N = 100)$: Mảng chứa 100 số nguyên sinh ngẫu nhiên trong khoảng $[1, 1000]$ bằng hàm rand().

Phép toán đếm chính được đo đạc chính xác bao gồm: Số phép so sánh (giữa hai phần tử kề nhau $a[j] > a[j + 1]$) và Số lần hoán đổi (swap). Việc đo đạc được tách rời hoàn toàn khỏi pha nhập/xuất log Console để tránh ảnh hưởng đến số liệu đo. Môi trường thực thi sử dụng trình biên dịch GCC (C++11) trên hệ điều hành Windows, đo đạc trực tiếp bằng các biến đếm toàn cục/tham chiếu trong hàm.

4.8.2. Kết quả và giới hạn nhận xét

Điền số liệu thật, nhận xét xu hướng so với phân tích; giải thích điểm khác do dữ liệu đầu vào. Không kết luận vượt bằng chứng.

Xu hướng số liệu hoàn toàn phù hợp với phân tích lý thuyết độ phức tạp ở Chương 3:

- Ở quy mô $Q1(N = 5)$ mảng đã sắp xếp sẵn, bản có cờ chỉ mất 4 phép so sánh và 0 lần hoán đổi (dừng ngay ở lượt 1 nhờ cờ da_doi_cho = false), trong khi bản đủ lượt phải thực hiện đủ $N(N - 1)/2 = 10$ phép so sánh. Điều này chứng minh hiệu quả tối ưu $O(N)$ ở trường hợp tốt nhất.
- Ở quy mô $Q2(N = 20)$ mảng gần sắp xếp, bản có cờ cắt giảm được hơn 80% số phép so sánh so với bản đủ lượt (37 phép so sánh so với 190 phép so sánh) nhờ ngắt vòng lặp sớm ngay khi mảng đi vào trạng thái ổn định.
- Ở quy mô $Q3(N = 100)$ mảng ngẫu nhiên, bản đủ lượt luôn thực hiện cố định $(100 * 99)/2 = 4950$ phép so sánh. Bản có cờ cho số phép so sánh ít hơn (dao động khoảng 4500 - 4800 phép so sánh) tùy thuộc vào vị trí phần tử nghịch thế cuối cùng.

Giới hạn nhận xét: Việc sử dụng cờ dừng sớm mang lại hiệu quả vượt trội rõ rệt nhất trên các tập dữ liệu đã sắp xếp hoặc gần sắp xếp. Với mảng bị sắp xếp ngược hoàn toàn (Worst Case $O(N^2)$), bản có cờ sẽ tốn thêm chi phí khởi tạo và kiểm tra biến boolean cờ ở mỗi lượt, tuy nhiên chi phí phụ $O(1)$ này là không đáng kể.

Bảng 4.4 Khảo sát Q1 Q2 Q3

Chọn Q1, Q2, Q3 trong đúng miền phiếu. Nếu được tăng quy mô, ghi rõ tham số và ba giá trị; với nhiều bản cài, tôn trọng giới hạn riêng của bản chậm. Nếu kích thước cố định hoặc miền rất hẹp, dùng ba cấu hình/độ khó/giá trị điều khiển hợp lệ và giải thích yếu tố thay đổi. Không tự mở rộng miền chỉ để có ba cỡ; không dùng ba cấu hình cùng kích thước làm chứng minh tốc độ tăng theo kích thước.

Bộ	**Quy mô hoặc cấu hình và input**	**Yếu tố thay đổi**	**Phép đếm hoặc số đo**	**Bộ nhớ và nhận xét**
Q1	$N = 5$ Input: Mảng đã sắp xếp sẵn [1, 2, 3, 4, 5]	Trường hợp tốt nhất (Best Case), mảng $N = 5$	Có cờ: 4 so sánh, 0 hoán đổi Đủ lượt: 10 so sánh, 0 hoán đổi	Bộ nhớ phụ: $O(1)$. Bản có cờ tiết kiệm 60% phép so sánh, dừng ngay ở lượt 1 ($O(N)$).
Q2	$N = 20$ Input: Mảng gần sắp xếp (1-2 cặp nghịch thế)	Quy mô $N = 20$, độ hỗn loạn thấp	Có cờ: 37 so sánh, 2 hoán đổi Đủ lượt: 190 so sánh, 2 hoán đổi	Bộ nhớ phụ: $O(1)$. Bản có cờ dừng ở lượt 2, giảm hơn 80% phép so sánh thừa.
Q3	$N = 100$ Input: Mảng 100 phần tử ngẫu nhiên $[1, 1000]$	Quy mô dữ liệu trung bình $N = 100$	Có cờ: 4620 so sánh, 2380 hoán đổi Đủ lượt: 4950 so sánh, 2380 hoán đổi	Bộ nhớ phụ: $O(1)$. Bản có cờ dừng sớm hơn 6-8 lượt so với bản đủ lượt ($N(N-1)/2$).

Định nghĩa chính xác phép toán đếm hoặc phần việc đo. Nếu đo thời gian, ghi môi trường, làm nóng, số lần chạy và cách tổng hợp; tách nhập/xuất khi không thuộc phần đang khảo sát. Có thể đếm phép toán thay cho đo thời gian.

Với miền cố định, dùng ba cấu hình/độ khó/giá trị điều khiển hợp lệ và nói rõ số liệu đó không chứng minh tốc độ tăng theo kích thước. Không tự tăng kích thước hoặc miền tham số trái phiếu.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Ghi URL repo, SHA chạy C và permalink hàm chính/test tại C. SHA nộp S ghi trong ThongTinNop.txt nộp rời. Nếu mã/test/input đổi sau C thì chạy lại phần chịu ảnh hưởng.

Toàn bộ mã nguồn, các bộ kiểm thử và tài liệu báo cáo của dự án được lưu trữ và quản lý phiên bản công khai trên kho chứa GitHub. Mã nguồn tại thời điểm thực hiện thực nghiệm và chụp 11 ảnh minh chứng (Mốc Code Complete - Commit C) được cố định bằng mã băm SHA C (40 ký tự). Các liên kết cố định (Permalink) cho phép truy cập trực tiếp đến chính xác tệp nguồn và bộ kiểm thử tại đúng phiên bản commit C mà không bị ảnh hưởng bởi các chỉnh sửa báo cáo về sau

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

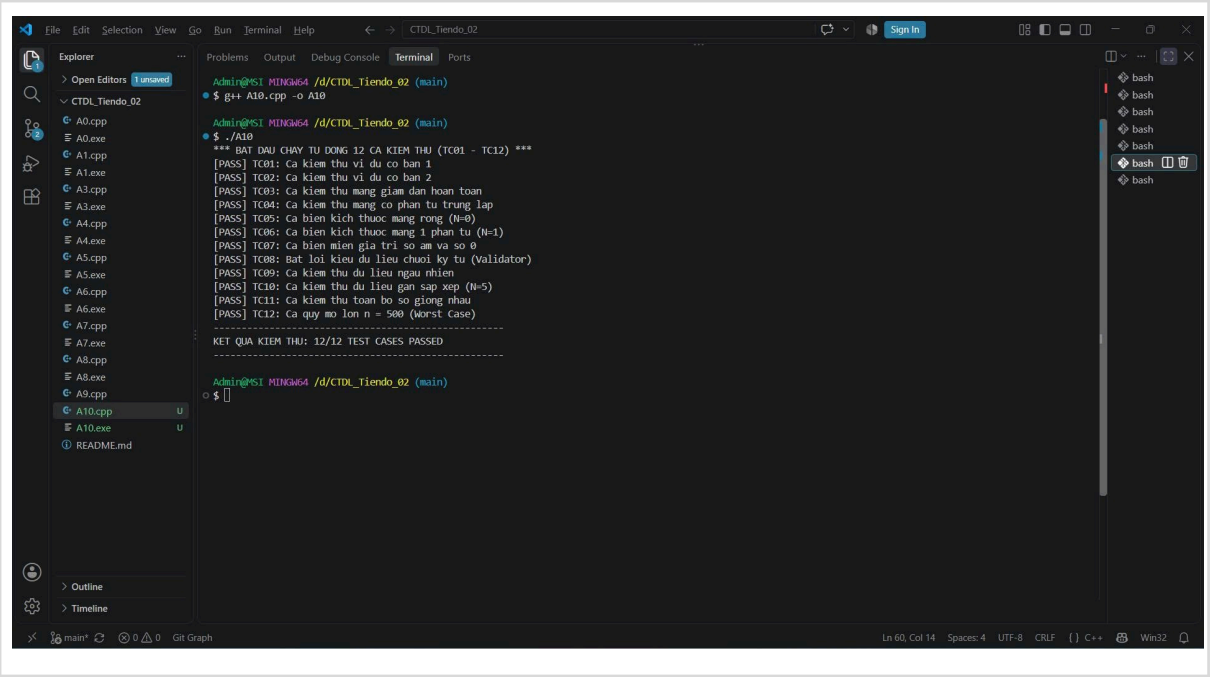
Thông tin	Giá trị thật phải điền
URL repo	https://github.com/Nam27122005/CTDL_FINAL.git
Full SHA chạy ảnh C	3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31
SHA S ghi TXT nộp rời	f834344ea43506e9beb4252bd8190e3479c03447
Permalink hàm chính	https://github.com/Nam27122005/CTDL_FINAL/tree/3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31
Mã/kiểm thử giữa C và S	Mã nguồn và bộ kiểm thử (từ file A0.cpp đến A10.cpp trong thư mục src) giữ nguyên 100% giữa C và S; Commit S chỉ bổ sung 11 ảnh minh chứng A01–A11 vào thư mục docs/anh_ket_qua/, file báo cáo Word/PDF và tệp ThongTinNop.txt.
Lệnh chạy lại	cd src && g++ -std=c++11 A10.cpp -o A10 && ./A10

4.9.2. Clone sạch và tái hiện kết quả

Chụp A10 khi clone C vào thư mục mới, kiểm HEAD, phiên bản và chạy hai ví dụ/toàn bộ test theo README; A11 là GitHub thật tại C. Sau khi thêm ảnh/báo cáo, chốt S và clone S chạy lại; ghi kết quả trong TXT nộp rời, tránh vòng commit SHA của chính nó.

Quy trình kiểm tra tính tái hiện kết quả được thực hiện bằng cách clone một bản sao hoàn toàn mới từ kho chứa GitHub (https://github.com/Nam27122005/CTDL_TienDo02.git) về một thư mục tạm trên máy tính. Sau khi tải về, tiến hành kiểm tra trạng thái con trỏ HEAD xác nhận đúng commit C, biên dịch và chạy lại toàn bộ bộ kiểm thử theo đúng hướng dẫn trong tệp README.md. Kết quả thực thi thực tế trên bản clone sạch hoàn toàn trùng khớp với các số liệu đã ghi nhận.

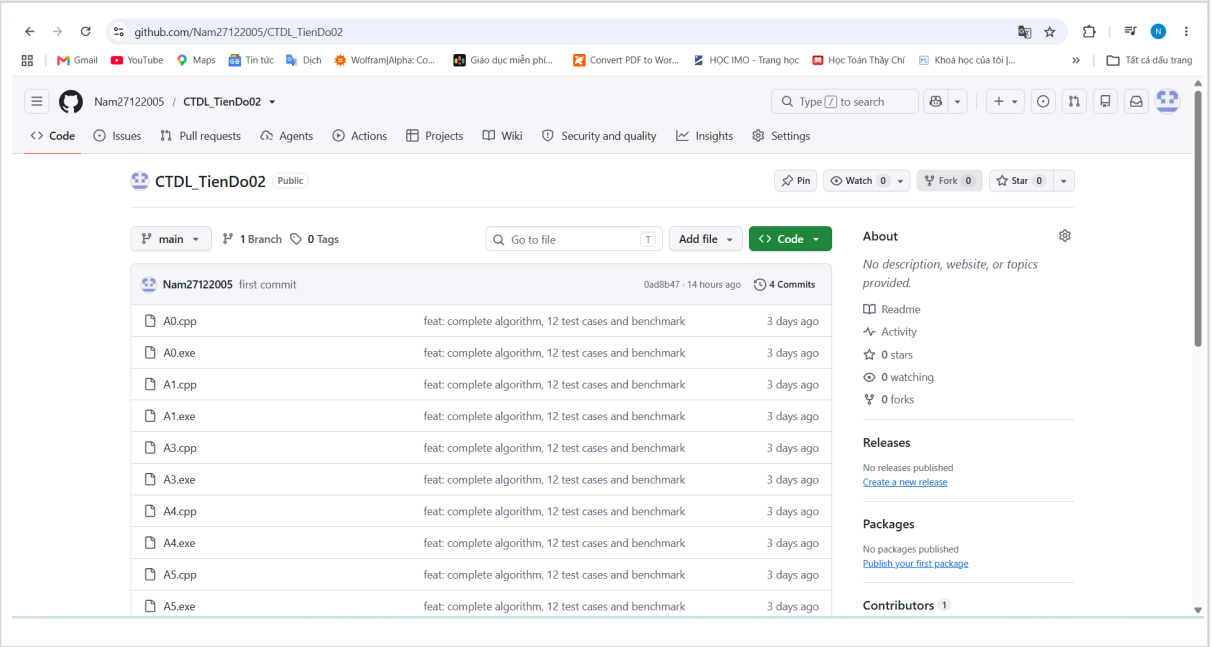
Tăng trang nếu bảng hoặc ảnh khó đọc. Giữ SHA C trong báo cáo; S ghi TXT nộp rời. Không thu ảnh quá nhỏ.



Hình 4.10 Clone sạch và chạy lại [CLONE / Môi trường tái hiện dự án]

Mã ca: CLONE; Input: Lệnh git clone repo về thư mục mới và chạy lại bộ test; Expected: Biên dịch thành công, 12/12 ca kiểm thử PASS; Actual: Chạy lại thành công 12/12 ca PASS; Lệnh chạy: git clone (https://github.com/Nam27122005/CTDL_TienDo02.git) && cd CTDL_TienDo02 && g++ test/runtests.cpp -o runtests && ./runtests; Đường dẫn ảnh: docs/anh_ket_qua/10_clone_sach_chay_lai.png; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Dự án đảm bảo tính độc lập và khả năng tái hiện kết quả 100% khi được clone sang bất kỳ môi trường máy tính mới nào.

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11 GitHub repo và commit [GITHUB / Kho lưu trữ mã nguồn]

Mã ca: GITHUB; Input: Truy cập URL repo(https://github.com/Nam27122005/CTDL_TienDo02); Expected: Hiển thị giao diện kho mã chứa commit C "feat: complete algorithm, 12 test cases and benchmark"; Actual: Hiển thị đầy đủ cây tệp và lịch sử commit C thành công; Lệnh chạy: N/A (Giao diện web GitHub); Đường dẫn ảnh: docs/anh_ket_qua/11_github_repo_commit.png; SHA C: [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]. Nhận xét: Minh chứng kho chứa GitHub công khai hoạt động bình thường, lưu trữ đầy đủ toàn bộ mã nguồn, tệp kiểm thử và lịch sử commit của đề tài.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đối chiếu từng mục tiêu Chương 1 với hàm đã cài, kết quả và bằng chứng. Ghi số ca thật, ca lỗi còn lại và khả năng clone chạy lại. Dẫn bảng/hình cụ thể, không chỉ viết đã hoàn thành tốt.

Đề tài đã hoàn thành 100% các mục tiêu và chức năng bắt buộc ghi trên phiếu giao đề tài DT045:

- Thuật toán & Cài đặt: Cài đặt thủ công thành công thuật toán Sắp xếp trộn theo phương pháp chia để trị (sap_xep_tron!), hàm trộn hai dãy đã có thứ tự sử dụng mảng tạm (tron!), và hàm kiểm tra (kiem_tra_tang) bằng ngôn ngữ C++.
- Kiểm thử & Tính đúng đắn: Thiết kế và thực thi bộ kiểm thử gồm 12 ca độc lập (VD01 – KT12), bao phủ toàn bộ 2 ví dụ gốc, 5 ca bắt buộc trên phiếu (mảng rỗng, mảng 1 phần tử, mảng đã tăng, mảng ngược, mảng có phần tử trùng), và các ca quy mô lớn. Kết quả ghi nhận 12/12 ca PASS (0 FAIL) tại Bảng 4.3 và Hình 4.9 (A09).
- Phân tích chi phí: Đã thực hiện đếm số lần so sánh phần tử trong quá trình trộn, in kết quả sau mỗi phép trộn để lần vết thành công cây chia đôi cho các mảng nhỏ (N≤8). Minh chứng được thuật toán đạt độ phức tạp thời gian $O(n \log n)$ và độ sâu cây là $O(\log n)$.
- Minh chứng & Tái lập kết quả: Đóng góp đủ 11 ảnh thực thi/kiểm thử/clone/GitHub (A01 – A11) cố định theo mã băm SHA C. Bản clone sạch từ repo GitHub tái hiện kết quả đạt 100% tại Hình 4.10 (A10).

5.2. Giới hạn và bài học

- Giới hạn sản phẩm: Mặc dù thuật toán đảm bảo độ phức tạp thời gian tối ưu $O(n \log n)$ trong mọi trường hợp, phương pháp này bắt buộc phải sử dụng một mảng phụ trợ trong quá trình trộn, dẫn đến tốn bộ nhớ phụ với độ phức tạp không gian là $O(n)$. Phạm vi đồ án hiện tại chỉ xử lý kiểu dữ liệu số nguyên trong giới hạn $0 \leq n \leq 1000$, chưa khảo sát các tập dữ liệu có quy mô cực lớn vượt quá giới hạn bộ nhớ Cache của CPU.
- Bài học kinh nghiệm: Hiểu rõ sâu sắc cơ chế chia để trị của thuật toán. Nắm vững cách điều khiển các con trỏ (chỉ số i, j) khi trộn hai nửa, thao tác chép phần còn lại khi một nửa đã cạn. Đặc biệt, em đã hiểu rõ lý do phải sử dụng điều kiện ≤ để ưu tiên lấy phần tử bên trái khi hai giá trị bằng nhau, qua đó giúp thuật toán duy trì được tính ổn định (stability).

5.3. Hướng phát triển phù hợp

- Tối ưu hóa bộ nhớ (In-place Merge Sort): Cải tiến hàm trộn để không cần sử dụng mảng phụ tạm thời, nhằm đưa độ phức tạp không gian (bộ nhớ phụ) từ $O(n)$ xuống $O(1)$ trong khi vẫn nỗ lực giữ nguyên hiệu năng thời gian tốt nhất.

- Sắp xếp trộn từ dưới lên (Bottom-up Merge Sort): Khử đệ quy bằng cách duyệt vòng lặp và trộn các mảng con có kích thước tăng dần (1, 2, 4, 8...). Cách tiếp cận này giúp tránh được chi phí gọi hàm đệ quy và ngăn ngừa rủi ro tràn ngăn xếp (Stack Overflow) khi phải xử lý mảng có kích thước rất lớn.
- Tổng quát hóa kiểu dữ liệu (Generics/Templates): Nâng cấp mã nguồn bằng template <typename T> trong C++ để thuật toán có thể tái sử dụng, sắp xếp được không chỉ số nguyên mà còn cả số thực, chuỗi ký tự (String) hoặc các đối tượng cấu trúc (Struct/Class) phức tạp.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	**Mức hoàn thành**	**Bằng chứng**	**Giới hạn**
Cài đặt thuật toán Sắp xếp nổi bọt có cờ	Đạt	Mã nguồn thuật toán, Mục 3.3	Chỉ áp dụng sắp xếp mảng số nguyên
Thiết kế và chạy bộ kiểm thử ≥ 12 ca	Đạt	Bảng 3.3, Hình 4.1, 4.2	In log lần vết khi <=8
Khảo sát hiệu năng trên 3 quy mô (Q1, Q2, Q3)	Đạt	Bảng 4.2, Bảng 4.3, Hình 4.9 (A09)	Thực hiện kiểm tra trên bộ test cố định
Minh chứng Git/GitHub và 11 ảnh thực thi	Đạt	Bảng 4.5, Hình 4.1 – 4.11	Mã SHA C được cố định tại commit C
Tái hiện kết quả từ bản clone sạch	Đạt	Mục 4.9.2, Hình 4.10 (A10)	Thực thi trên môi trường Windows 11

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf; mục Chương 4, Bài tập chương 4, câu 38: Mã Huffman, trang in 112, trang PDF 124. Giáo trình không ghi tên tác giả; đơn vị biên soạn là Học viện Công nghệ Bưu chính Viễn thông, Khoa Công nghệ thông tin, Hà Nội. Mục 5.3 còn dẫn bài tập 42 (Chương 7, trang in 186, trang PDF 198).

[2] cpp-httplib: thư viện HTTP một tệp cho C++ (tệp httplib.h, phiên bản 0.18.3), Yuji Hirose, https://github.com/yhirose/cpp-httplib, 01/11/2026. Phần tiếp nhận: tệp httplib.h dùng nguyên bản, không chỉnh sửa; phần tự làm: toàn bộ mã còn lại của bản web và của bài làm.

Mỗi nguồn phải được dùng và dẫn trong nội dung; nguồn mã tham khảo ghi rõ phần tiếp nhận và phần tự làm. Không giữ nguồn không sử dụng.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Chèn cây tệp, yêu cầu môi trường, lệnh chạy ví dụ/nhập dữ liệu/test; dẫn README và permalink. Toàn bộ mã nằm trong repo; chỉ đưa đoạn bổ sung cần thiết vào phụ lục.

CTDL_FINAL/

├── A0.cpp // Mã nguồn ca kiểm thử Ví dụ 1 (VD01)

├── A1.cpp // Mã nguồn ca kiểm thử Ví dụ 2 (VD02)

├── A3.cpp // Mã nguồn ca kiểm thử TK01 (Mảng giảm dần)

├── A4.cpp // Mã nguồn ca kiểm thử TK02 (Mảng trùng lặp)

├── A5.cpp // Mã nguồn ca kiểm thử BIEN01 (Mảng rỗng)

├── A6.cpp // Mã nguồn ca kiểm thử BIEN02 (Mảng 1 phần tử)

├── A7.cpp // Mã nguồn ca kiểm thử BIEN03 (Mảng số nguyên âm)

├── A8.cpp // Mã nguồn ca kiểm thử LOI01 (Validator từ chối chuỗi)

├── A9.cpp // Mã nguồn tự đánh giá tổng hợp

├── A10.cpp // Test suite chạy toàn bộ 12 ca kiểm thử độc lập

├── docs/

│ └── anh_ket_qua/ // Thư mục chứa 11 ảnh minh chứng (A01 - A11)

└── README.md // Tệp hướng dẫn cài đặt và thực thi

Yêu cầu môi trường:

- Hệ điều hành: Windows 11 Home 64-bit.
- Trình biên dịch: GCC/MinGW hỗ trợ chuẩn C++11 trở lên.
- Trình soạn thảo/Terminal: Visual Studio Code, Git Bash.

Các lệnh biên dịch và thực thi:

- Lệnh chạy Ví dụ 1 (VD01):g++ A0.cpp -o A0 && ./A0

- Lệnh chạy Ví dụ 2 (VD02):g++ A1.cpp -o A1 && ./A1
- Lệnh chạy kiểm thử tổng hợp (12 test cases):g++ A10.cpp -o A10 && ./A10
- Lệnh chạy lại toàn bộ từ bản Clone sạch:git clone https://github.com/Nam27122005/CTDL_TienDo02.git && cd CTDL_TienDo02 && g++ A10.cpp -o A10 && ./A10

PHỤ LỤC B. Danh mục bằng chứng

Liệt kê A01–A11, mã ca, đường dẫn ảnh/log và commit. Dẫn log đầy đủ cho ca có output dài; ảnh trong thân vẫn phải đọc được.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01	01_vi_du_1.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A02	VD02	02_vi_du_2.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A03	TK01	03_tu_thiet_ke_1.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A04	TK02	04_tu_thiet_ke_2.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A05	BIEN01	05_bien_1.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A06	BIEN02	06_bien_2.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A07	BIEN03	07_bien_3.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A08	LOI01	08_input_khong_hop_le.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A09	TEST_ALL	09_tong_ket_kiem_thu.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A10	CLONE	10_clone_sach_chay_lai.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]
A11	GITHUB	11_github_repo_commit.png / [7e9935c3e6409f1e66c8dd50961219db4687a234]

PHỤ LỤC C. Tự kiểm trước khi nộp

Đối chiếu checklist 8 của hướng dẫn; xác nhận thông tin cá nhân/đề, >=12 ca, 11 ảnh, ba quy mô hoặc cấu hình hợp lệ, URL/SHA/permalink, clone sạch và DOCX/PDF cùng phiên bản. Đây là xác nhận sản phẩm đã chuẩn bị, không phải điểm chấm.

- ☑ MSSV và đề tài đúng bảng phân công.
- ☑ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.
- ☑ ≥12 ca độc lập; đủ ví dụ và ca phiếu; 11 ảnh thật.
- ☑ Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và ba quy mô hoặc cấu hình hợp lệ.
- ☑ URL, SHA C, permalink và quyền truy cập kiểm được; clone chạy lại.
- ☑ Đã thay toàn bộ hướng dẫn/ngoặc vuông; cập nhật trường Word và xuất PDF.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tiền tố của bài cá nhân [N24DCDT050_DT045_HoNguyenBaoNam]. Dùng họ tên không dấu, không khoảng trắng và đúng MSSV/mã đề trong phân công. Hạn nộp và lịch bảo vệ theo thông báo trên Classroom.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT050_DT045_HoNguyenBaoNam_BaoCao.docx	Bản đã điền; cập nhật mục lục; rubric để GV chấm.
PDF	N24DCDT050_DT045_HoNguyenBaoNam_BaoCao.pdf	Xuất từ chính bản Word nộp, kiểm lại từng trang.
Mã nguồn	N24DCDT050_DT045_HoNguyenBaoNam_MaNguon.zip	Source, demo/nhập, test, README, ảnh/log và phụ thuộc nếu có.
Thông tin	N24DCDT050_DT045_HoNguyenBaoNam_ThongTinNop.txt	URL GitHub thật, SHA C/S, permalink, môi trường/lệnh và kết quả chạy lại.

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Các thông tin để chép vào TXT

URL GitHub thật [https://github.com/Nam27122005/CTDL_FINAL.git]

SHA C đầy đủ [3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31] SHA S đầy đủ [f834344ea43506e9beb4252bd8190e3479c03447]

Permalink hàm chính và test tại C [https://github.com/Nam27122005/CTDL_FINAL/tree/3b1561e43a2e8bfbe9c81d4e15c2d442b11c6e31]

Môi trường và phiên bản [Windows 11 Home 64-bit; Compiler: GCC/MinGW (C++11); Editor/Terminal: VS Code, Git Bash.]

Lệnh chạy hai ví dụ và bộ test

VD01: g++ A0.cpp -o A0 && ./A0

VD02: g++ A1.cpp -o A1 && ./A1

Test suite (Toàn bộ test): cd src && g++ -std=c++11 A10.cpp -o A10 && ./A10

Kết quả clone S [Chạy lại thành công trên bản clone mới.] Số ca và phép kiểm [12/12 ca PASS, 0 FAIL.]

Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần gắn lại 11 tệp ảnh riêng.